Versión pre-oficial

INSTITUTO TECNOLÓGICO AUTÓNOMO DE MÉXICO



PROGRAMACIÓN GENÉTICA EN LA PLANIFICACIÓN DE TEMPORADAS DE BÉISBOL

TESIS

QUE PARA OBTENER EL TÍTULO DE

INGENIERA EN COMPUTACIÓN

PRESENTA

LESLIE PAMELA BRENES VALENCIA

ASESOR: DR. ANDRÉS GÓMEZ DE SILVA GARZA

CIUDAD DE MÉXICO 2026

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tdp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

«Con fundamento en los artículos 21 y 27 de la Ley Federal del Derecho de Autor y como titular de los derechos moral y patrimonial de la obra titulada "PROGRAMACIÓN GENÉTICA EN LA PLANIFICACIÓN DE TEMPORADAS DE BÉISBOL", otorgo de manera gratuita y permanente al Instituto Tecnológico Autónomo de México y a la Biblioteca Raúl Bailléres Jr., la autorización para que fijen la obra en cualquier medio, incluido el electrónico, y la divulguen entre sus usuarios, profesores, estudiantes o terceras personas, sin que pueda percibir por tal divulgación una contraprestación.»

LESLIE PAMELA BRENES VALENCIA

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/tesis-itam

FECHA

FIRMA

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Papá Cuau, Mamá Bety, ¡lo logramos!

Dreams don't happen overnight, you have to be awake. (Coach Prime).

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Agradecimientos

Agradezco el apoyo de mi mamá Beatriz, mi papá Cuauhtémoc, mis hermanas Graciela y Romina, mi esposo Jeffrey y su papá Marty Dickinson, mis tíos Teresa, Minerva, Antonio, Javier, mis primos Omar y Javi, y demás miembros de nuestra familia y amigos porque siempre me hacen sentir bienvenida, comprendida y amada. Agradezco el apoyo y enseñanzas de mi asesor, el Dr. Andrés, durante mi maratón en el ITAM. De manera general, agradezco a la madre Tierra, la ciencia, el internet y pertenecer a la era de la transformación digital.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tfg

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Resumen

El béisbol es uno de los deportes favoritos en muchos países, especialmente en Estados Unidos. La ubicación de los juegos (en casa o de visita) determina la logística, la ruta y la frecuencia de traslado de los equipos. En consecuencia, generar calendarios de temporada es un desafío logístico que requiere considerar múltiples restricciones, haciendo indispensable el uso de un programa computacional. Se usará como referencia la Grandes Ligas de Béisbol (del inglés *Major League Baseball*) (MLB) para presentar ejemplos y ofrecer un marco teórico sobre las necesidades de este tipo de sistemas en la vida real.

Al buscar minimizar la distancia total de traslado entre sedes, observamos que, aunque existen programas comerciales para generar calendarios deportivos, pocos proveen la opción de optimizar esta restricción en particular.

Este diseño ingenieril propone una arquitectura de software para generar calendarios de temporadas de béisbol que minimice la distancia total mediante la aplicación del paradigma de la Programación Genética (PG). Este método, perteneciente al campo de la Inteligencia Artificial

© Leslie Pamela Brenes Valencia
Tesis IIMAS 2026 · github.com/lespam/koza-tsp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

(IA), emplea principios biológicos adaptados a programas de computadora y se diferencia de la optimización tradicional en que no requiere convertir el problema a complejas funciones matemáticas continuas.

En este paradigma, una solución se expresa mediante un árbol de sintaxis jerárquico que representa la selección de fechas y equipos que conforman los juegos del calendario.

El resultado es un prototipo funcional implementado a través de una aplicación web dinámica. Para evaluar su rendimiento, se comparó el proceso evolutivo contra la generación de calendarios de forma aleatoria. Por un lado, dado un número fijo de juegos y series, se crearon calendarios mediante la selección aleatoria de elementos. Por el otro, se implementó la PG, siguiendo los principios de Koza (1992), utilizando la librería DEAP. Estos principios establecen que los programas pueden generar otros programas iterativamente para encontrar mejores soluciones en el espacio de búsqueda.

Los resultados de la experimentación revelaron que los operadores genéticos genéricos provistos por la librería cambiaban drásticamente la estructura de las ramas (fechas y equipos), lo que frecuentemente rompía la restricción fundamental de la cantidad de juegos requeridos por equipo. Esto obligó a reorientar la función de aptitud para priorizar un filtro de calendarios válidos sobre la minimización de la distancia.

Al someter a competencia los calendarios válidos generados aleatoriamente (población inicial) con aquellos resultantes de las operaciones genéticas, se observó que la solución final seleccionada como el óptimo local nunca provino del proceso de evolución profundo. Debido a

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/espan/koza_ttp
Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

los métodos de selección elitista, el calendario final siempre fue una de las soluciones iniciales aleatorias.

Este resultado (donde la generación aleatoria superó a la evolución algorítmica de la PG) se atribuye a limitaciones de diseño en los operadores de cruzamiento y mutación, los cuales no estaban adaptados a las particularidades logísticas del intercambio de juegos.

La contribución principal de este trabajo es la construcción exitosa de una plataforma de optimización genérica capaz de procesar y evaluar las características específicas de una liga de béisbol. Como trabajo futuro, se recomienda desarrollar operadores genéticos específicos al contexto deportivo para refinar soluciones iniciales válidas, abriendo la puerta a incorporar nuevas restricciones que hoy en día son un desafío monumental, incluso para la MLB.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozo_ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Índice general

1. Introducción **1**

1.1. Identificación del problema 4

1.2. Objetivo 8

2. Análisis del problema **11**

2.1. Marco teórico 12

2.1.1. Logística deportiva 12

2.1.2. Medición geoespacial 15

2.1.3. Optimización computacional 17

2.1.4. Programación genética (PG) 21

2.1.5. Problema de Transportación de Torneo (PTT) y trabajos relacionados 25

2.2. Definición de la función objetivo 26

2.3. Selección del método de optimización 27

2.3.1. Enfoque de investigación de operaciones 27

2.3.2. Enfoque de la PG (Seleccionado) 27

2.4. Alcance 28

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

2.5. Requerimientos y Restricciones 28

3. Diseño **32**

3.1. Diseño del programa evolutivo 33

3.1.1. Pasos preparatorios 34

3.2. Arquitectura 42

3.2.1. Modelos para la base de datos 47

3.2.2. Módulos desarrollados 48

3.3. Estándares utilizados 51

4. Implementación **53**

4.1. Aplicaciones 53

4.1.1. Programa genético 55

4.1.2. Vistas y Modelos 57

4.1.3. Parámetros de la implementación 58

4.2. Especificaciones de software y hardware 60

5. Resultados **61**

5.1. Descripción de las pruebas 61

5.2. Análisis de resultados 64

5.3. Generador de calendarios sin la PG 66

5.4. Generador de calendarios con la PG 69

5.5. Desafíos y aprendizajes 70

6. Conclusiones **73**

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Apéndices **77**

I. Análisis de distintas ligas profesionales de béisbol 77

I.1. Liga Mexicana de Béisbol (LMB) 77

I.2. Liga Mexicana del Pacífico (LMP) 79

I.3. Liga Venezolana de Béisbol Profesional (LVBP) . . . 81

I.4. Nippon Professional Baseball league (NPB) 84

II. Análisis de modificaciones en el calendario de la MLB . . . 86

II.1. Modificaciones de la liga 87

II.2. Huelgas de jugadores y eventos externos 87

II.3. Cambios en las reglas y formato 88

III. Variación en el número de juegos de la MLB 89

Glosario **91**

Referencias **93**

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttb

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Índice de cuadros

2.1. Requerimientos 29
2.2. Restricciones 30

3.1. Primitivas y terminales del programa 39
3.3. Ejemplos del cálculo de la aptitud 46

4.1. Definición de variables de entrada y parámetros de salida . . 56
4.2. Resumen de la implementación 59

5.1. Parámetros de entrada y salida 65
5.2. Parámetros y resultados del experimento de optimización genética 66
5.3. Evaluación de valores de aptitud por generación (Búsqueda Aleatoria) 68
5.4. Parámetros y resultados del experimento de optimización genética 69
5.5. Resultados de aptitud y juegos incompletos a lo largo de las generaciones (PG) 70

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tts

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

III.1. Relación de juegos por temporada en la MLB (1974-2023) . 89

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Índice de figuras

1.1. Sesgo visual en la comparación de trayectorias (MLB 2016) 6
1.2. Mapa de desplazamientos en 2020 de la MLB 9
1.3. Mapa de la temporada 2023 de la MLB 10

2.1. Modelo de la superficie de la Tierra 16
2.2. Algoritmo genético convencional 23
2.3. Paradigma de la programación genética 24

3.1. Diagrama de clases 47
3.2. Diagrama de eventos 49
3.3. Diagrama de Interfaces 50

5.1. Calendario 72

I.1. Mapa de la Liga Mexicana de Béisbol 78
I.2. Calendario de la temporada 2022 de la LMB 79
I.3. Mapa de la Liga Mexicana del Pacífico 80
I.4. Calendario de la Liga Mexicana del Pacífico 2019 81
I.5. Mapa de la Liga Venezolana de Béisbol Profesional 82

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

I.6. Calendario semanal para la Liga Venezolana de Béisbol Profesional 83
I.7. Mapa de la liga de béisbol profesional de Japón 85
I.8. Calendario de la liga de béisbol profesional de Japón. Agosto 2019 86
III.9. Gráfica de la distancia total por año de la MLB 90

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Índice de algoritmos

3.1. Método que representa un programa solución 38
3.2. Método para la inicialización aleatoria 44
3.3. Cálculo de la aptitud de un programa que representa un calendario 45

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tts

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Capítulo 1

Introducción

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/espam/koza-ttp

> El deporte tiene el poder de cambiar el mundo … tiene el poder de inspirar. Tiene el poder de unir a la gente de una manera que pocas otras hacen. Le habla a la juventud en un idioma que comprenden. El deporte puede crear esperanza donde antes sólo había desesperación. Es más poderoso que los gobiernos para romper las barreras raciales.
>
> Nelson Mandela

Cada año, el calendario de juegos de la temporada para las Grandes Ligas de Béisbol (MLB, del inglés *Major League Baseball*) afecta las vidas y el estado de ánimo de millones de personas. Durante el periodo comprendido entre 2009 y 2019, la asistencia promedio por juego fue aproximadamente de 30 mil personas (Gough, 2021). La temporada 2024, la cadena de televisión FOX registró 1.841 millones de espectadores (en promedio) y la cadena ESPN reportó un promedio de 1.740 millones (MLB.com, 2025).

Gerentes, jugadores, fanáticos y periodistas son afectados por varios

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

factores como el rendimiento deportivo, los contratos, las negociaciones laborales y las tendencias mediáticas. Sin embargo, no mucha gente sabe cómo es que se establece el calendario de la MLB y menos aún tienen idea de lo que conlleva.

La temporada regular de la MLB se compone de conjuntos de juegos llamados serie. Una serie de béisbol es un conjunto de juegos que se disputan entre dos equipos durante un periodo determinado, por lo regular de 2 a 4 juegos en la misma semana. El patrón de juego en casa o de visita es importante porque determina las fechas en que los estadios venderán boletos. A diferencia de otras disciplinas, el béisbol no se rige por un formato estrictamente equilibrado (*round-robin*), sino que la cantidad de enfrentamientos varía según los criterios de la MLB en cada temporada. El número regular de juegos por equipo es 162, pero un año (2020) fue de 60.

La distancia involucrada en el transporte de equipos de la MLB se extiende de la costa este a oeste de Estados Unidos, y de Canadá a la punta de Florida. Los equipos realizan traslados constantes, alternando continuamente sus estancias como locales y visitantes.

Desde 1981, la MLB lanzó la convocatoria para la generación del calendario donde se solicitaron estrictas pautas y reglas. Por mucho tiempo, estas no pudieron ser satisfechas por los programas computacionales existentes. En su lugar, durante 25 años se delegó esta tarea a la empresa *Stephenson Systems*, quienes lo hacían simplemente usando lápiz y papel (Borzi, 2009).

Holly Stephenson es una matemática graduada de Cornell y su esposo Henry cuenta con una maestría en arquitectura. Juntos conforman la

© Leslie Pamela Branes Valencia
Tesis ITAM 2026 · github.com/espanlkoz-tfg

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

empresa *Stephenson Systems*, quienes inicialmente desarrollaron un sistema informático, pero, al igual que los otros participantes, su programa no cumplía con los requisitos de la MLB. Después de analizar calendarios de temporadas anteriores, confrontaron a la MLB con evidencia de que las restricciones y reglas, que en teoría eran imprescindibles, no habían sido cumplidas. Así, los Stephenson consiguieron su primer contrato para programar 2,430 juegos jugados por 30 equipos en 180 días. Sobre las restricciones de la MLB, Henry Stephenson mencionó que, en última instancia, cambiaron sus servicios de tratar de escribir un programa que generara calendarios a convertirse en un servicio de mediación encargado de generar convenios, reglas y acomodaciones solicitadas por las partes involucradas (Crowe, 1987). Este dúo prosperaría hasta que se optó por un software de generación aleatoria.

No es casualidad que el investigador Michael Trick, especialista en investigación de operaciones, se encuentre adscrito al *Sports Scheduling Group* (SSG), un grupo dedicado a la programación deportiva que logró convencer a la MLB de que utilizando las matemáticas se podría producir un calendario más justo (Borzi, 2009). Entre las publicaciones académicas de Trick destaca el artículo sobre el Problema de Transportación de Torneo (PTT), traducido del inglés *Traveling Tournament Problem* (TTP) (Easton, Nemhauser, y Trick, 2001).

El Problema de Transportación de Torneo es un problema de optimización inspirado en la logística de la MLB, cuyo objetivo es minimizar la distancia de viaje cumpliendo con el patrón de juegos en casa y de visita permitiendo a la comunidad científica modelarlo y comparar

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozatto

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

distintos resultados. Este es un problema de optimización y de viabilidad que invita a explorar distintos métodos para ser resuelto.

Dado que no siempre se pueden cubrir todas las peticiones, planear calendarios es un desafío. Es comprometerse a encontrar una solución posiblemente imperfecta. Por ejemplo, Holly mencionó que uno de los equipos de Nueva York siempre tuvo una larga lista de solicitudes especiales permitidas por la liga (Wright, 2017).

Finalmente, en 2010, Katy Feeney, vicepresidenta de Programación y Relaciones con Clubes, confirmó que la MLB se encargaría de producir internamente el calendario (Borzi, 2009). Desde entonces, no se ha obtenido información más detallada sobre el proceso, pues este se encuentra a cargo del comisionado de la MLB.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozo-tto

1.1. Identificación del problema

Uno de los aspectos más complejos en la organización de torneos es la optimización de la distancia entre los equipos participantes.

Un calendario con desplazamientos excesivos podría dificultar la asistencia de los aficionados, y afectar el rendimiento de los jugadores, ya que implica más cansancio por *jet lag* y cambios de zona horaria. Además, tiene consecuencias para el medio ambiente por emisiones de CO_2 de acuerdo con Rameez Naqvi et al. (2022).

Por lo tanto, se identifica que existe el siguiente problema: integrar el componente de distancia a un programa para generar calendarios de torneos.

Como se ilustra en la comparativa de la figura 1.1, la mala

Copia de preservación · Prohibida su reproducción sin citar la fuente

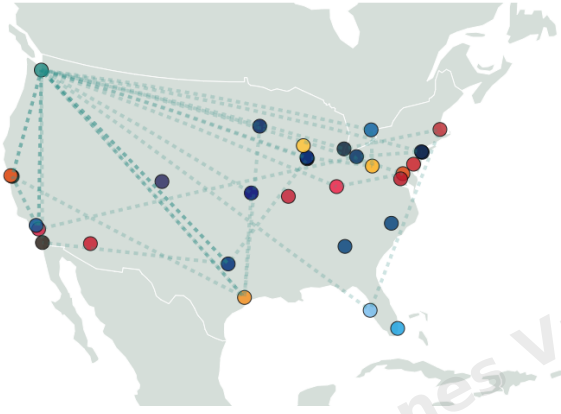
calendarización puede generar disparidades críticas: mientras que un equipo puede tener una ruta compacta, otro puede verse forzado a recorrer casi el doble de distancia bajo el mismo número de juegos. Este desgaste logístico real, evidenciado en la diferencia de más de 23,000 millas entre los equipos analizados, justifica la búsqueda de métodos de optimización más precisos.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

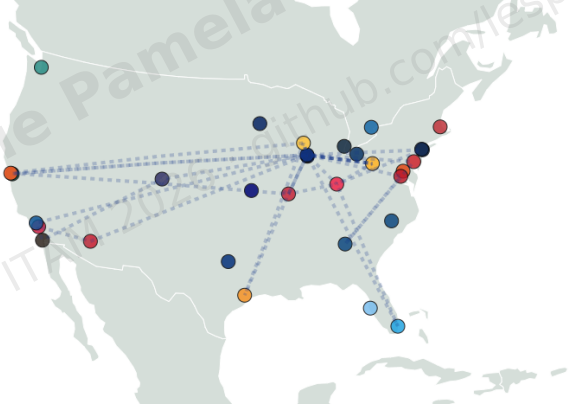
Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 1.1. Comparación de rutas 2016: Sesgo entre percepción visual y distancia real.



(a) Marineros de Seattle



(b) Cachorros de Chicago

El total de millas viajadas por *Los Marineros* de Seattle fue de 47,649 mi (76,683.63 km), mientras que el de *Los Cachorros* de Chicago fue de casi la mitad (24,243 mi, 39,015.32 km). Imágenes tomadas de Baseball Savant (2023).

En consecuencia, dado el carácter dinámico del calendario de béisbol y

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

los cambios históricos en la estructura de la MLB, el programa de optimización desarrollado debe poseer la flexibilidad necesaria para adaptarse a variaciones rápidas en la cantidad de juegos por temporada.

Para modelar una liga genérica se escogió a la MLB. ¿Es realista pensar que una liga puede cambiar su estructura? Primero, tras examinar su historia, existen dos casos donde la estructura del calendario cambió drásticamente. El primero es el confinamiento por COVID-19, como se muestra en el mapa de la figura 1.2 y el segundo es la temporada de 2023, donde se realizaron un par de series fuera de Estados Unidos y se aplicó la regla donde los equipos juegan al menos una vez contra todos los otros equipos de ambas ligas. El mapa de los viajes durante esta temporada se encuentra en la figura 1.3.

Comprender qué tipo de cambios se han dado es crucial para entender los desafíos de generar calendarios optimizados y que puedan ser adaptados por distintos usuarios. Para más detalles, véase el apéndice III con un resumen de cambios en la historia de la MLB.

Además de la optimización del calendario de una liga existe otro caso a considerar, el generar calendarios equilibrados. Las variaciones en los viajes realizados por cada equipo pueden tener un impacto significativo en los costos administrativos y el desempeño de los equipos (Rameez Naqvi et al., 2022). Cuando estas decisiones son arbitrarias pueden generarse desventajas. Este contexto justifica el diseño de un programa computacional que incorpore métodos de optimización y se adapte a las restricciones del usuario.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozo_ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

1.2. Objetivo

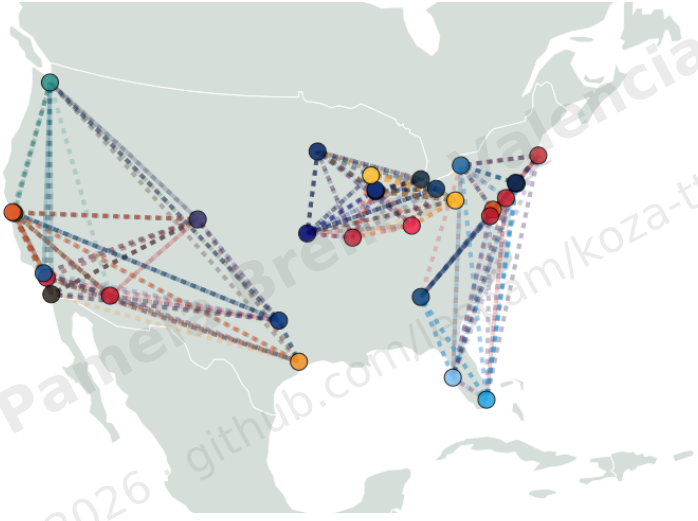
El objetivo general de este proyecto es desarrollar un prototipo funcional (prueba de concepto) configurable que automatice la generación de calendarios de torneos deportivos, tomando como base instancias de la MLB, para minimizar la distancia total de traslado. Como objetivo específico, se evaluará el desempeño del paradigma de la programación genética frente a resultados obtenidos mediante búsquedas aleatorias establecidas como punto de referencia.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 1.2. Mapa de desplazamientos en 2020 de la MLB



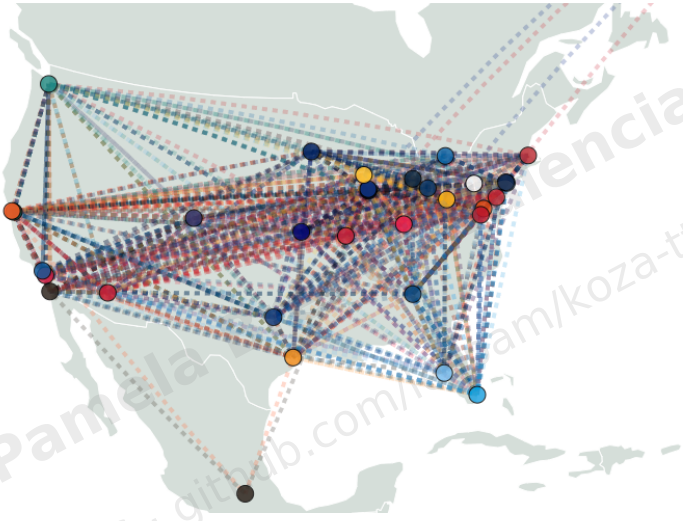
Para cada punto el color representa ubicaciones de distintas sedes. En 2020 por la emergencia sanitaria COVID-19, se priorizó minimizar el movimiento de los equipos y sus fanáticos. Se estableció que los equipos jugaran contra rivales en su misma división geográfica solamente (Wilson, 2020). El 6 de julio se anunció el calendario, reduciendo el número de juegos por equipo de 162 a 60. Fuente: Baseball Savant (2023).

© Leslie Pamela Breña Valencia
Tesis TAM 2026 · github.com/lpbv/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 1.3. Mapa de la temporada 2023 de la MLB



En 2023 cada equipo se debe enfrentar al menos una vez contra todos los demás equipos. Se aumentó los juegos entre ligas dándole menos peso a juegos de la misma división. Todos los fanáticos vieron a sus equipos enfrentarse a estrellas como Shohei Ohtani y Juan Soto. Además, como parte de una gira, en México se enfrentaron *Los Padres* de San Diego y *Los Gigantes* de San Francisco mientras que en Inglaterra se enfrentaron *Los Cachorros* de Chicago y *Los Cardenales* de San Luis. Fuente: Baseball Savant (2023).

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Capítulo 2

Análisis del problema

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

En este capítulo se realiza un análisis del problema general para la generación de calendarios de torneo. Si bien ya existe software comercial, la minimización de distancia entre las sedes de los equipos no es una funcionalidad común.

Por lo tanto, este capítulo busca establecer cuáles son las características y limitaciones necesarias para diseñar un prototipo funcional.

A continuación, se revisa el artículo Problema de Transportación de Torneo (PTT) de Easton et al. (2001) para modelar los métodos de optimización que se han usado para minimizar la distancia. El artículo del PTT formula una estructura académica para estudiar la minimización de la distancia total recorrida por un grupo de equipos al visitar distintos estadios durante un torneo, cumpliendo con restricciones de longitud. Este problema tiene similitud con el Problema del Vendedor Viajero (PVV) (en inglés, *Traveling Salesman Problem* (TSP)) ya que en ambos se busca

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

minimizar la distancia, con la diferencia de que uno trata de un vendedor viajero y el otro de un grupo de equipos viajeros.

Luego, se comparan las fortalezas y limitaciones de algunos algoritmos de optimización para seleccionar aquel que se considere más adecuado para este proyecto. Se definirá un marco teórico que permita dar mayor contexto a estas consideraciones. Después, en la sección de trabajos relacionados, se muestran aplicaciones de ligas en diferentes países que citan el PTT. Finalmente, se presentan las secciones de alcance, requerimientos y restricciones consideradas para el diseño final de la solución.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

2.1. Marco teórico

Para facilitar la comprensión de este trabajo, los conceptos del marco teórico se han dividido en tres áreas fundamentales: la terminología de logística deportiva, los modelos de medición geoespacial y los enfoques de optimización computacional.

2.1.1. Logística deportiva

Definición 2.1.1. ***Equipos participantes.*** *E_i es el equipo número* i*, desde 1 hasta* n *equipos participantes, donde* n *es par.*

Definición 2.1.2. ***Liga.*** *Es un grupo de equipos.*

Definición 2.1.3. ***Torneo.*** *Un conjunto de enfrentamientos entre equipos de una liga en un periodo fijo. Cada equipo comienza y termina el torneo en su lugar de origen.*

Copia de preservación · Prohibida su reproducción sin citar la fuente

Definición 2.1.4. ***Fecha de juego.*** *Una fecha de juego f es una posible fecha para la asignación de juegos de un torneo. f_1 es la fecha de inicio y f_m es la fecha fin:*

$$f_1, f_2, \ldots, f_m$$

Definición 2.1.5. ***Juego de béisbol.*** *Un juego es una tupla compuesta de tres elementos: fecha de juego y dos equipos distintos a enfrentarse, E_A y E_B. Se representa como (f, E_A, E_B), que significa* el juego donde el equipo A visita al equipo B en la fecha f. *Un equipo juega en un solo periodo a la vez y ningún equipo puede enfrentarse a sí mismo.*

$$juego = (f, E_A, E_B)$$

Definición 2.1.6. ***Serie.*** *Un conjunto de juegos que se disputan entre dos equipos durante un periodo determinado. Por lo regular, en la MLB, las series constan de 2 a 4 juegos en la misma semana.*

Definición 2.1.7. ***Viaje.*** *Un conjunto de series consecutivas fuera de casa.*

Definición 2.1.8. ***Estancia.*** *Un conjunto de series consecutivas en casa.*

Definición 2.1.9. Longitud. *El número de viajes o estancias consecutivas.*

Definición 2.1.10. ***Representación de ubicaciones.*** *Dos ubicaciones, A y B (con sus respectivas coordenadas de latitud y longitud), se representan como $A = (x_A, y_A)$ y $B = (x_B, y_B)$.*

Definición 2.1.11. ***Distancia de traslado.*** *$d(A, B)$ representa la distancia de traslado desde la sede A hacia el estadio del equipo B.*

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Para calcular la distancia de traslado es necesario conocer la ubicación de las sedes, las cuales se obtienen a partir de una base de datos proporcionada por el usuario.

Definición 2.1.12. ***Simetría de distancia.*** $d(A, B) = d(B, A)$. *Se asume que la distancia de traslado entre dos equipos es la misma sin importar cuál sea el visitante.*

Para traducir la logística deportiva a un modelo computacional, es necesario definir la estructura de datos que albergará los eventos:

Definición 2.1.13. ***Lista.*** *Estructura de datos computacional que permite almacenar una colección de elementos de forma ordenada. En este trabajo, se denota con corchetes* `[]`.

Definición 2.1.14. ***Calendario.*** T *representa una lista de juegos correspondientes al calendario de temporada de una liga de béisbol. Todos los equipos se enfrentan un número de veces previamente establecido. Los juegos son únicos y están ordenados cronológicamente. Donde* m *representa el número total de juegos y se asume que para cada juego existe una fecha correspondiente.*

$$T = [juego_1, juego_2, juego_3, \dots juego_m]$$

Definición 2.1.15. ***Número esperado de juegos.*** v_0 *representa la cantidad exacta de juegos que cada equipo debe enfrentarse durante la temporada regular.*

Definición 2.1.16. ***Distancia de viaje individual.*** *Dados* E_A *y* E_B *dos equipos distintos, la distancia de viaje entre ellos en una fecha*

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespamkoz-tto

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

específica f_k se denota con el subíndice correspondiente como $d(E_A, E_B)_{f_k}$. Esta distancia es constante en cualquiera de las fechas disponibles:

$$d(E_A, E_B)_f = d(E_A, E_B)$$

Definición 2.1.17. ***Distancia de viaje de un equipo.*** *$D(E_A)$ representa la distancia de viaje total del equipo E_A. Esto equivale a la suma de todos los recorridos realizados por dicho equipo durante el torneo:*

$$D(E_A) = \sum_{i \neq A}^{n} \sum_{k=1}^{m} d(E_A, E_i)_{f_k}$$

Definición 2.1.18. ***Distancia total.*** *La distancia total del calendario, $D(T)$, es igual a la suma de las distancias recorridas por todos los equipos participantes:*

$$D(T) = \sum_{i=1}^{n} D(E_i)$$

© Leslie Pamela Briones Valencia
Tesis MAM 2026 · github.com/lespam/kobra-ttp

2.1.2. Medición geoespacial

Para modelar una función objetivo es importante considerar que la superficie de la Tierra no es plana, sino que tiene forma de esferoide. Se puede producir un modelo geométrico tridimensional o el plano cartesiano bidimensional. Existen diferentes modelos para calcular la distancia entre dos puntos sobre la Tierra; la figura 2.1 ilustra estos modelos geométricos.

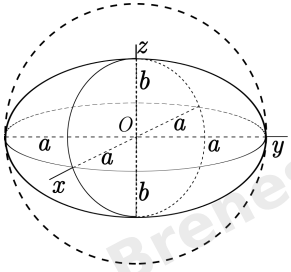
Definición 2.1.19. ***La distancia euclídea.*** *Es la fórmula utilizada para medir la distancia recta entre dos puntos. Las unidades utilizadas (por ejemplo, kilómetros) pueden introducir sesgos cuando se compara la distancia de líneas rectas contra trayectos reales si no se realizan ajustes*

Copia de preservación · Prohibida su reproducción sin citar la fuente

para adecuarse al modelo tridimensional de la Tierra (Grootendorst, 2021).

$$d(A,B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}.$$

Figura 2.1. Modelo de la superficie de la Tierra



El elipsoide, la esfera y el plano cartesiano sirven como referencias para escoger una función que calcule la distancia. El punto O representa el origen. Los ejes x, y y z permiten representar ubicaciones en dos o tres dimensiones. En el modelo elipsoide se tienen dos semiejes iguales de longitud a y un tercer semieje b. Fuente: (Mercator, 2022)

© Lesia Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Definición 2.1.20. ***La norma del semiverseno.*** *También conocida en inglés como fórmula* haversine*, es una medida utilizada para calcular la distancia entre dos puntos en una superficie esférica. Esta medida es comúnmente empleada en aplicaciones de Sistemas de Información Geográfica (GIS) (Upadhyay, 2015).*

$$d(A,B) = 2 \cdot \sin^{-1}\left(\sqrt{\sin^2\left(\tfrac{x_2 - x_1}{2}\right) + \sin^2\left(\tfrac{y_2 - y_1}{2}\right) \cdot \cos(x_1) \cdot \cos(x_2)}\right) \cdot \mu_E,$$

donde μ_E representa el radio medio de la Tierra (6,371.009 km).

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Definición 2.1.21. ***Fórmulas de Vincenty.*** *Método iterativo, desarrollado por T. Vincenty, para calcular la distancia y el acimut entre dos puntos tomando en cuenta el achatamiento de los polos y la forma real de la Tierra.*

Aplicar las fórmulas de Vincenty permite obtener resultados más precisos, pero su implementación es más compleja debido a su naturaleza iterativa. Se ha demostrado que pueden lograr un error máximo de 0.5 mm en el acimut (Vincenty, 1975). En comparación, distancias de alrededor de 10,000 km tendrían un margen de error de aproximadamente 30 km usando el método del semiverseno. Por lo tanto, las fórmulas de Vincenty son más adecuadas cuando se requiere una precisión extrema, mientras que métodos más simples ofrecen resultados aceptables con menor esfuerzo computacional (Baskar y Anthony Xavior, 2021).

2.1.3. Optimización computacional

Definición 2.1.22. ***Métodos de optimización.*** *Métodos que aplican modelos matemáticos, análisis estadístico, algoritmos o heurísticas para obtener la mejor solución de un conjunto de soluciones posibles. Dentro de estos métodos, una heurística es una técnica basada en la experiencia que busca soluciones prácticas y aceptables, aunque no necesariamente perfectas.*

Definición 2.1.23. ***Optimización clásica.*** *Consiste en realizar alguna transformación analítica de la función para determinar los puntos de interés. Por ejemplo, el método de los multiplicadores de Lagrange o el método de*

© Leslie Pamela Prenes Valencia
Tesis | M.M 2026 | github.com/lesprem/Kepler-tfg

Copia de preservación · Prohibida su reproducción sin citar la fuente

Newton para encontrar raíces de una función (Albert, Bautista, Vicenç, y María, 2016).

Definición 2.1.24. ***Programación matemática.*** *Consiste en un problema donde se formula una función objetivo y se describen las restricciones a optimizar; por ejemplo, $z = f(x)$ donde E es una región factible dentro del universo de soluciones y $x \in E$. La programación lineal, la entera, la binaria y la investigación de operaciones entran en esta categoría.*

Dentro de la programación matemática existen métodos, como el algoritmo simplex o el descenso de gradiente, que aplican estos conceptos para encontrar máximos o mínimos.

Definición 2.1.25. ***Métodos de metaheurísticas.*** *Se denomina heurística a las técnicas basadas en la experiencia o analogías útiles para resolver un problema. El prefijo "meta-" indica que tales heurísticas son aplicables a problemas genéricos. Aunque no garantizan la solución óptima, suelen proporcionar soluciones aceptables para problemas analíticamente inabordables. Ejemplos de esto son la programación evolutiva o la búsqueda tabú (Voß, 2000).*

Definición 2.1.26. ***Investigación de operaciones.*** *Disciplina que aplica modelos matemáticos, análisis estadístico y algoritmos para optimizar recursos y estrategias (Quiroga, 2020). Se encuentra en el conjunto de los métodos de programación matemática y es aplicable en logística, ingeniería y ciencia.*

© Lesly Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tfg

Copia de preservación · Prohibida su reproducción sin citar la fuente

Definición 2.1.27. ***Programación lineal.*** *Se trata de problemas en los que tanto la función objetivo como las restricciones son lineales. Se resuelven comúnmente con el método simplex, un algoritmo iterativo que mejora la solución en cada paso hasta llegar al óptimo (Albert et al., 2016).*

Definición 2.1.28. ***Programación entera.*** *Problemas en los que algunas o todas las variables deben tomar valores enteros. Requiere métodos como el de ramificación y acotamiento (*branch-and-bound*) (Salazar López, 2019).*

Definición 2.1.29. ***Programación no lineal.*** *Problemas en los que la función objetivo o alguna restricción no es lineal. Son más complejos debido a la posible existencia de múltiples óptimos locales (Albert et al., 2016).*

Definición 2.1.30. ***Programación dinámica.*** *Técnica donde la solución óptima depende de decisiones tomadas en etapas anteriores, descomponiendo el problema en subproblemas recursivos (Kariuki, 2023).*

Definición 2.1.31. ***Inteligencia artificial.*** *Conjunto de técnicas metaheurísticas que combinan una búsqueda guiada por resultados prometedores. La Inteligencia Artificial (IA) permite abordar problemas que no se pueden modelar con métodos tradicionales por su dinamismo o incertidumbre (Takeyas, 2007) (Luger y Stubblefield, 1989).*

La IA permite abordar problemas como el reconocimiento facial o la evolución de programas. Algunas técnicas como la búsqueda aleatoria o el recocido simulado ayudan a mejorar el rendimiento de algoritmos, pero carecen de la adaptación dinámica o inspiración biológica propia de la IA avanzada.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Para comprender el paradigma de la programación evolutiva, primero es necesario definir los elementos biológicos y computacionales que estructuran esta metaheurística:

Definición 2.1.32. ***Gen.*** *Representa una característica codificada (por ejemplo, un color representado por bits). El gen está compuesto por* ***alelos****, que son las distintas combinaciones posibles de bits.*

Definición 2.1.33. ***Individuo.*** *Representa una solución única dentro del espacio de búsqueda.*

Definición 2.1.34. ***Población.*** *Es el conjunto de individuos o posibles soluciones en una generación determinada.*

Definición 2.1.35. ***Criterio de terminación.*** *Condición que detiene el proceso evolutivo (tiempo, número de generaciones o convergencia).*

Definición 2.1.36. ***Función de aptitud (*fitness function*).*** *La función que evalúa qué tan bien se desempeña cada individuo en su contexto para resolver el problema. En este trabajo la aptitud representa el grado de viabilidad y adecuación de un individuo, pero un alto nivel de aptitud no garantiza matemáticamente que dicha solución sea la óptima (el mínimo o máximo global absoluto del problema).*

Definición 2.1.37. ***Operadores genéticos.*** *Operaciones de reproducción, cruzamiento y mutación que permiten la exploración del espacio de soluciones. ELa reproducción clona individuos con una probabilidad p_r, el cruzamiento intercambia segmentos de código con una*

© Leslie Pamela Briones Valencia
Tesis ITAM 2025 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

probabilidad p_c, y la mutación altera aleatoriamente un bit con una probabilidad p_m.

Definición 2.1.38. ***Algoritmos genéticos.*** *Un Algoritmo Genético (AG) es una metaheurística que imita el mecanismo de la naturaleza para evolucionar y adaptar especies. En su libro* Adaptación en sistemas naturales y artificiales*, Holland (1992) describe cómo una población de individuos sobrevive y se reproduce basándose en su aptitud, contemplando cruzamientos y mutaciones de ADN.*

2.1.4. Programación genética (PG)

A diferencia de los AG, que operan sobre cadenas binarias fijas, la programación genética (PG) manipula directamente programas de computadora.

Definición 2.1.39. ***Programación genética.*** *La Programación Genética (PG) aplica el enfoque de los AG a programas de computadora. Comienza con una población de programas de inicialización aleatoria que, mediante operaciones genéticas, generan las estructuras necesarias para producir un programa solución.*

En la PG, los programas se representan como árboles de sintaxis, donde los nodos internos son operadores y los externos (hojas) son operandos.

Definición 2.1.40. ***Terminales y primitivas.*** *Se llaman terminales a las variables de entrada. Las funciones primitivas son el conjunto de métodos (aritméticos, lógicos) que permiten la construcción de nuevos elementos mediante su combinación.*

© Leslie Pamela Briones Valencia
Tesis MAM 2026 · github.com/lespam/koza-ttp
Copia de preservación · Prohibida su reproducción sin citar la fuente

Definición 2.1.41. ***Condición de cierre.*** *Se satisface cuando cualquier función puede usar como entradas los resultados de otras funciones o terminales del conjunto (J. R. Koza, 1998).*

Definición 2.1.42. ***Aridad.*** *El número de entradas que un método requiere para operar.*

Definición 2.1.43. ***Búsqueda en paralelo.*** *La reproducción de múltiples programas con poblaciones distintas a lo largo de generaciones actúa como un algoritmo de búsqueda en paralelo (J. Koza, 1992).*

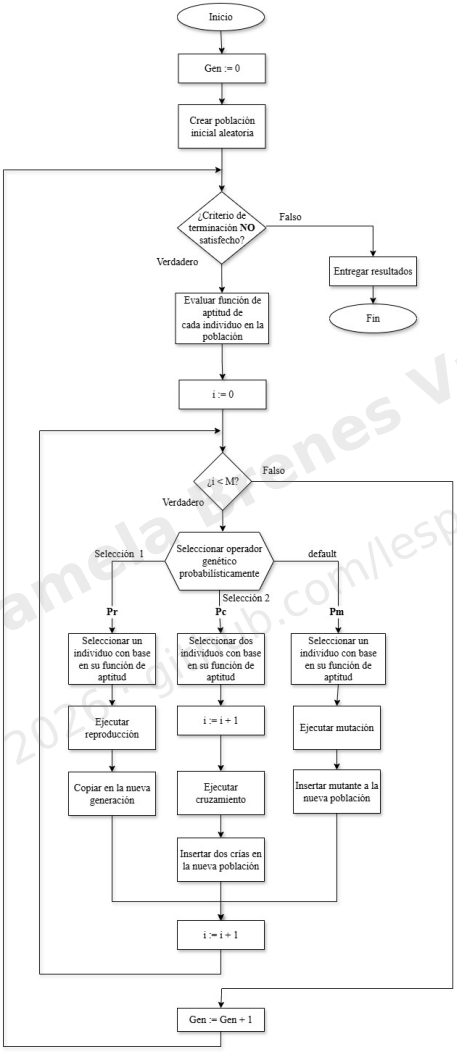
La PG procesa estructuras jerárquicas y dinámicas en forma y tamaño, permitiendo manipular subexpresiones (subárboles) de programas padres. El diagrama del proceso se encuentra en la figura 2.3.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lospam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 2.2. Algoritmo genético convencional



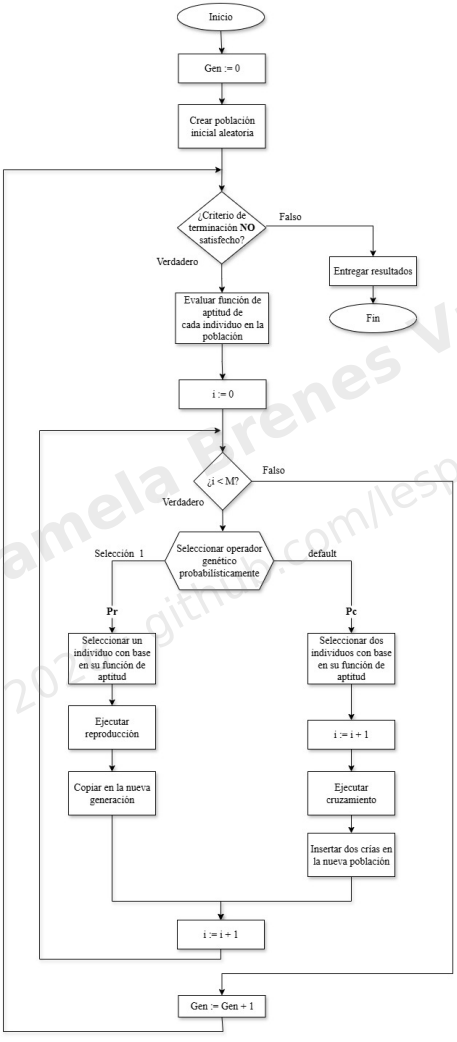
En un AG convencional una cadena de bits representa un individuo. *Gen* es la generación, i es un contador y M el tamaño de la población. Los operadores de reproducción p_r, cruzamiento p_c y mutación p_m dependen de la adaptación al problema. Fuente: Koza, 1992

© Leslie Pamela Ynes Valencia
Tesis ITAM 2026 github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 2.3. Paradigma de la programación genética



En la PG un programa representa un individuo solución. Los operadores principales son reproducción p_r y cruzamiento p_c. Fuente: Koza, 1992.

© Leslie Pamela Prenes Valencia
Tesis ITAM 2021 github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

2.1.5. Problema de Transportación de Torneo (PTT) y trabajos relacionados

El PTT se presentó por primera vez en el artículo de Easton et al. (2001). Se asume que hay n equipos (par) en un torneo de todos contra todos a doble vuelta.

Definición 2.1.44. ***Problema de Transportación de Torneo (PTT).*** *Es el problema de optimización que busca un calendario que satisfaga un conjunto de restricciones de viajes consecutivos, minimizando la distancia total recorrida por todos los equipos.*

Entrada técnica del PTT:

1. Un conjunto de n equipos $E = \{E_1, \ldots, E_n\}$.
2. Una matriz de distancia simétrica D $(n \times n)$, donde cada elemento corresponde a la distancia de traslado entre dos equipos $d(E_i, E_j)$.
3. Parámetros enteros de longitud de viajes l, u (usualmente $l = 1$ y $u = 3$).

Easton, Nemhauser, y Trick (2002) propusieron un algoritmo de ramificación y precio (*branch-and-price*), donde cada columna representa el recorrido de un equipo. Debido a la complejidad, inicialmente se considera solo un subconjunto de variables y se evalúa en paralelo mediante *strong-bound* (ramificación fuerte). Además, incorporaron una *primal heuristic* que asegura que si un equipo visita, su oponente juegue en casa, limitando a un máximo de 2 equipos por recinto.

© Leslie Pamela Brenes Valencia
Tesis IIMAS 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

2.1.5.1. Estado del software actual

Como se identificó previamente, programas existentes como *LeagueLobster*, *PrintYourBrackets* y *Playpass* automatizan la gestión de torneos, pero no consideran la ubicación geográfica de los estadios. Solo *TourneyMachine* ofrece una opción de minimización de distancia, pero bajo un modelo de pago por suscripción (Rameez Naqvi et al., 2022).

2.1.5.2. Aplicaciones en otras ligas

En Corea, Kim (2019) generó una metaheurística con programación entera para la Liga de la Organización Coreana de Béisbol (del inglés *Korea Baseboll Organization*) (KBO), demostrando que su modelo es superior a la programación manual en términos de distancia y equidad, aunque requirió más de 100 horas de cómputo. En Cuba, Almira, Rodríguez, Suárez, y González (2021) utilizó una biblioteca de clases que resuelve problemas de optimización utilizando metaheurísticas (BiCIAM) para reducir en un 6.79 % los kilómetros recorridos en la Serie Nacional, logrando tiempos de ejecución inferiores a 5 minutos mediante el método de escalador de colinas.

© Leslie Pamela Arenes Valencia
Tesis ITAM 2026 · github.com/lespam/kuva-tdp

2.2. Definición de la función objetivo

El prototipo de este proyecto debe adaptarse a ligas genéricas. Se consideraron dos alternativas para evaluar la aptitud del programa generador:

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

1. **Minimizar la distancia total:** Buscar el calendario con la mínima distancia absoluta entre todos los generados.

   $$\text{mín}\, X, \text{ donde } X \text{ es la distancia de un calendario.}$$

2. **Minimizar la varianza:** Buscar que los calendarios generados sean consistentemente cercanos a la media de calidad.

Para este trabajo se seleccionó la distancia total, ya que el objetivo primordial es la optimización logística.

2.3. Selección del método de optimización

Se evaluaron dos perspectivas:

2.3.1. Enfoque de investigación de operaciones

Se basa en definir variables de decisión x_{ij} (número de encuentros) y una función objetivo lineal. Sin embargo, Thielen y Westphal (2011) demostraron que cuando se restringen los juegos consecutivos a 3, el problema es NP-duro. Esto significa que no existe un algoritmo para encontrar la solución óptima en tiempo finito (polinomial) conforme crece el número de equipos.

2.3.2. Enfoque de la PG (Seleccionado)

La PG es conveniente porque:

- Maneja la complejidad del espacio de búsqueda sin requerir exploración exhaustiva.

© Leslie Pamela Briones Valencia
Tesis TAM2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

- Permite una codificación natural: los calendarios se ven como programas jerárquicos de eventos que pueden intercambiarse.
- Es flexible ante cambios dinámicos (fechas, nuevas sedes) mediante la función de aptitud `calcula_distancia(calendario)`.

En este proyecto se utilizará la biblioteca Algoritmos evolutivos distribuidos en Python (del inglés *Distributed Evolutionary Algorithms in Python*) (DEAP) para implementar las operaciones evolutivas en Python.

2.4. Alcance

El prototipo se asume bajo las siguientes condiciones:

- El número de equipos es siempre par.
- Los equipos juegan las mismas series en los mismos periodos.
- No se consideran enfrentamientos dobles (*doubleheaders*) en esta etapa.
- Los estadios tienen ubicaciones fijas.

2.5. Requerimientos y Restricciones

Se detallan en las tablas 2.1 y 2.2. Destaca el cálculo de distancia mediante el semiverseno y el requerimiento de hardware (32 GB RAM) para soportar las múltiples poblaciones del algoritmo genético.

© Lesly Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Tabla 2.1. Requerimientos

Requerimiento	Descripción
Ingresar equipos	El prototipo acepta una lista de equipos y valida que el número de equipos es par.
Almacenamiento de la base de datos de estadios	El prototipo permite ingresar la información de la ubicación de los equipos con el nombre del equipo, nombre del estadio y coordenadas geográficas de su ubicación en formato decimal.
Sistema métrico	El prototipo trabaja con el sistema métrico internacional usando como medida los kilómetros y usa el formato de fecha año-mes-día.
Programación de juegos	El prototipo genera un calendario usando dos parámetros: fecha de inicio y número de juegos por equipo y número de juegos en una serie.
Generación del calendario	El prototipo genera un calendario que incluye la fecha y los contrincantes de cada juego. Es claramente distinguible qué equipo actúa como local y como visitante.
Cálculo de la distancia	El prototipo es capaz de calcular la distancia entre los estadios de los equipos usando la técnica del semiverseno. La distancia total se calcula mediante la suma total de la distancia de traslado realizados por los equipos visitantes. Esta distancia se considera una vez por juego y se asume que los equipos siempre regresan a casa.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozo-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Requerimiento	Descripción
Distancia total de la temporada	El prototipo muestra la distancia total recorrida de ida durante la temporada. La distancia total se calcula mediante la suma total de traslados realizados por equipos visitantes. Las cantidades presentadas como totales deben ser multiplicadas por dos si se desea obtener la distancia de traslado de ida y regreso.
Condición de paro	El prototipo debe poder ejecutarse en un tiempo razonable, debe evaluar múltiples diseños, y se espera que el usuario realice varias solicitudes para generar propuestas de calendarios.
Interfaz de usuario	El prototipo cuenta con una interfaz donde un usuario puede ingresar la información y ver el resultado.

© Leslie Pamela Prenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Tabla 2.2. Restricciones

Restricción	**Descripción**
Consistencia en el número de juegos	El sistema debe garantizar que cada equipo complete el número exacto de juegos programados (v_0). Se define como una restricción crítica para la viabilidad del calendario y cualquier desviación (diferencial $\neq 0$) un calendario inválido o incompleto.
El número de equipos	El número de equipos debe ser par.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Restricción	Descripción
No hay viajes consecutivos	Este prototipo considera que un equipo regresa a casa después de jugar fuera.
Fechas	La fecha fin debe coincidir con el último día de juegos programados.
Las fechas de la temporada serán proporcionadas por los organizadores.	No está dentro del alcance de este proyecto generar o adaptar reglas específicas para la asignación de fechas. Además, no se considerarán los juegos dobles. Se asume que otro sistema generará la lista de fechas contemplando las reglas y restricciones de la liga.
No hay juegos de un solo equipo	Ningún equipo puede tener un juego programado consigo mismo.
Sistema de coordenadas	La ubicación de los estadios debe ser descrita mediante coordenadas de latitud y longitud.
Se limita a una serie por semana	No se permitirá que un equipo tenga más de una serie programada en la misma semana.
No se considerarán restricciones adicionales	Otras restricciones mediante funciones de penalización o criterios que favorezcan las soluciones más adecuadas pueden ser adaptadas en el futuro.
Tiempo limitado	Dado que el problema puede tener múltiples soluciones, el procesamiento computacional debe ser limitado.
Equipo requerido para la ejecución	Windows 11 de 64 bits, 32 GB de RAM, procesador de 8 núcleos a 3.7 GHz, y tener instalado Python 3.7. También se requiere un teclado, ratón y monitor para operar el sistema.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2025 · github.com/lespam/kaiza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Capítulo 3

Diseño

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

El contenido de este capítulo se organiza de la siguiente manera: la sección 3.1 describe el programa evolutivo y propone la interacción con el usuario mediante una interfaz HTML. Se siguen los cinco pasos preparatorios definidos por Koza. Se define el conjunto de terminales $\mathcal{T}$ (operandos) y el conjunto de funciones primitivas $\mathcal{F}$ (operadores) que conformarán los programas de computadora (calendarios) y que se representan mediante árboles de sintaxis. Luego, se define el método para la inicialización y el cálculo de la aptitud de cada calendario.

En la sección 3.2 se describe la arquitectura del programa. Se presentan los diagramas UML y una descripción detallada de los módulos y modelos de la base de datos a desarrollar. Finalmente, en la sección 3.3 se mencionan los estándares utilizados.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

3.1. Diseño del programa evolutivo

Se eligió un programa genético por su capacidad teórica para abordar problemas complejos de optimización sin necesidad de una traducción matemática estricta a ecuaciones lineales. El problema del PTT adaptado a nuestro proyecto se formula de acuerdo al algoritmo 3.1.

Se empleó Python y la biblioteca DEAP por ser diseñada específicamente para la construcción de programas evolutivos. Python puede integrarse con otros paquetes y herramientas, incluyendo marcos de desarrollo para su despliegue en aplicaciones o páginas web, como es el caso de Django. La idea es desarrollar de forma independiente la funcionalidad de la interfaz del usuario.

La biblioteca DEAP tiene un papel central en la arquitectura porque proporciona estructuras de datos, operadores genéticos y herramientas que buscan abstraer gran parte de la complejidad del diseño, permitiendo centrarse en la representación de individuos y la codificación del problema. Esto permite mantener una separación clara entre la lógica del problema y los mecanismos evolutivos. Para estructurar los individuos y la población, se hace uso de los módulos `creator` y `base`. El módulo `creator` permite instanciar dinámicamente la función de aptitud (define la cantidad de objetivos, una ponderación de peso del objetivo mediante un número que cuando es positivo busca maximizar y cuando es negativo se busca minimizar) y la clase del individuo (define el objeto y lo encapsula en una clase). Por su parte, el módulo `base` proporciona la clase `Toolbox`, que actúa como un contenedor donde se registran algunos métodos auxiliares,

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/K-za-tq

Copia de preservación · Prohibida su reproducción sin citar la fuente

las funciones de inicialización, los operadores genéticos del algoritmo y el registro de resultados.

Dado que la PG requiere una estructura flexible para representar calendarios de longitud variable, se adopta la Lista como el tipo de dato fundamental. Esto permite que los nodos del árbol de sintaxis manipulen secuencias de juegos y evita la concatenación redundante de listas vacías, facilitando las operaciones de cruzamiento y mutación.

Asimismo, una interfaz web en Django permite visualizar el programa y validar la interoperabilidad.

3.1.1. Pasos preparatorios

Para usar el paradigma de la programación genética según J. Koza (1992), existen cinco pasos preparatorios. Estos son:

a) Escoger un conjunto de terminales,

b) Escoger un conjunto de funciones primitivas,

c) Definir una función de aptitud,

d) Definir los parámetros requeridos para la ejecución del programa, y

e) Escoger el método para evaluar los resultados y el criterio de terminación de la ejecución.

Si un calendario se representa mediante un árbol de sintaxis, entonces los nodos representan el orden de selección de fechas y equipos para crear una lista de juegos.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

3.1.1.a. Escoger un conjunto de terminales

Las terminales son variables con valores constantes (cuando el nodo tiene un valor asignado) o dinámicos (cuando el nodo representa un objeto cuyo valor es calculado mediante un método).

Primero, deben crearse los siguientes métodos: *fecha(lista)* para escoger una fecha aleatoriamente, y *equipo(lista)* para escoger un equipo aleatoriamente.

Luego, se proponen las siguientes terminales:

- Variables dinámicas: *getFecha* y *getEquipo* (solicitan valores aleatorios a los métodos *fecha(lista)* y *equipo(lista)*).
- Constantes: Múltiples elementos *fecha* y *equipo* (tantos como elementos en las listas), y la constante `[ ]` (lista vacía) como contenedor inicial.

© Leslie Pamela Briones Valencia
Tesis MAM 2026 · github.com/lespam/kozaltp

Las terminales propuestas se pueden expresar como:
$\mathcal{T} = \{getFecha, getEquipo, \texttt{[ ]}, f_1, f_2, \ldots, f_n, E_1, E_2, \ldots, E_m\}$.

Ejemplo. Un programa (calendario) posible es:

`[(getFecha, “TEX”, getEquipo)]`.

que al ejecutarse asigna valores como:

`[(“2025-10-14”, “TEX”, “WAS”)]`.

Esto representa un juego programado para el 14 de octubre de 2025, donde los Astros de Texas juegan contra los Nacionales de Washington.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

En resumen, las terminales representan los elementos básicos para construir un programa que genera un calendario de juegos como se puede ver en la tabla 3.1.

3.1.1.b. Escoger un conjunto de funciones primitivas

Las primitivas definen la estructura de las ramas mediante operaciones que transforman o combinan terminales, donde cada nodo puede ser un operando u operador. Este mecanismo permite enlazar las operaciones de forma iterativa.

La condición de cierre se cumple, ya que las funciones operan principalmente sobre tipos Lista y String, permitiendo entradas válidas para otros nodos.

Luego, se proponen las siguientes primitivas:

- El método *crear_juego* para construir juegos válidos y empaquetar una tupla de 4 elementos en una lista.
- El método *obtener_calendario* que construye un calendario mediante la concatenación de listas.
- Selección de una fecha (cadena de texto).
- Selección de un equipo (cadena de texto).

La función *crear_juego* actúa como el validador principal: si recibe cadenas de texto inválidas, genera un juego aleatorio para asegurar la integridad del calendario.

Se pueden expresar como:

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

$$\mathcal{F} = \{fecha, equipo, crear_juego, obtener_calendario\}.$$

Ejemplo. Un programa (calendario) posible es:

`[crear_juego([(“2025-10-09”, “TEX”, “WAS”)]`, “2025-10-10”, `equipo(“BOS”)`, `equipo(getEquipo()))]`.

que al ejecutarse asigna valores como:

`[`(“2025-10-09”, “TEX”, “WAS”), (“2025-10-10”, “BOS”, “LAD”)`]`.

Por lo tanto, cada primitiva y terminal puede representar un programa funcional, siempre que se respete la aridad de los operadores y la compatibilidad de los tipos de datos, permitiendo la reutilización de resultados entre operaciones, tal como se detalla en la tabla 3.1.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lesplam/K-za-tp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Algoritmo 3.1: Método que representa un programa solución

Nombre : *programa_genetico()*

Precondiciones: El número esperado de juegos v_0, una lista de equipos $L_E = (E_1, E_2, \ldots, E_n)$ con n par, las coordenadas de las sedes, un método para calcular su distancia y una lista de fechas $L_F = (f_1, f_2, \ldots, f_m)$.

Entrada : Conjunto inicial de fechas y equipos seleccionados por el usuario.

Salida : Un calendario de juegos en formato de lista, que minimice la distancia y el diferencial en el número de juegos de todos los equipos.

Sea C la función que evalúa la aptitud. C_1 evalúa el error en el número esperado de juegos y C_2 la distancia total. Se busca:

1. Minimizar el error en el número de juegos por equipo mediante la maximización de la diferencia entre v_0, el número esperado, y v_i, el observado. El valor óptimo es igual a cero.

$$C_1(T) = \text{máx}\left(-\sum_{i=1}^{n} |v_0 - v_i|\right)$$

2. Minimizar la distancia total recorrida por todos los equipos.

$$C_2(T) = \text{mín}\left(\sum_{i=1}^{n} D(E_i)\right)$$

© Leslie Pamela Brenes Valencia
Tesis IPN-UAM 2026 · github.com/lespam/kaza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Tabla 3.1. Primitivas y terminales del programa

Categoría	Elemento	Entradas	Salidas	Aridad
Terminal	`getFecha`	N/A	String	0
	`getEquipo`	N/A	String	0
	$f_1, \ldots, f_n$	N/A	String	0
	$E_1, \ldots, E_n$	N/A	String	0
	`[ ]`	N/A	Lista	0
Primitiva	`fecha`	String	String	1
	`equipo`	String	String	1
	`crear_juego`	lista (Lista), fecha (String), E_1, E_2 (String)	Lista	4
	`obtener_calendario`	$lista_1$, $lista_2$ (Lista)	Lista	2

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 | github.com/lespam/proyecto

Copia de preservación · Prohibida su reproducción sin citar la fuente

3.1.1.c. Definir una función de aptitud

Se propone una función de aptitud multiobjetivo, $C(T) = eval_calendario(T)$, con las siguientes dos metas: primero, garantizar que el número de juegos sea el correcto y, segundo, minimizar la distancia total. El método se encuentra descrito en el cuadro de código del algoritmo 3.3.

3.1.1.d. Definir los parámetros requeridos para la ejecución del programa

La ejecución requiere definir el tamaño de la población (POP), la probabilidad de cruzamiento (CXPB), la probabilidad de mutación (MUTPB) y el número máximo de generaciones (NGEN). Aunque no existe una regla fija para definir estos valores, para la ejecución de este prototipo se establecieron empíricamente los siguientes valores base:

- Un tamaño de población inicial de calendarios asignados aleatoriamente (M) igual a 100.
- Una probabilidad de cruzamiento (CXPB) de 0.8.
- Una probabilidad de mutación (MUTPB) de 0.2.

En el contexto de los árboles de sintaxis, el cruzamiento se realiza intercambiando subárboles entre dos calendarios padre, lo que equivale a permutar bloques de juegos o series completas. La mutación altera un nodo específico de forma aleatoria, sustituyendo, por ejemplo, un equipo o

© Leslie Pamela Briones Valencia
Tesis | IAM 2026 · github.com/leslipam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

una fecha en un juego particular. Se eligió una probabilidad de cruzamiento alta (CXPB = 0.8) para favorecer la convergencia rápida mediante la herencia de buenas estructuras logísticas, y una mutación baja (MUTPB = 0.2) para mantener la diversidad genética sin destruir las soluciones que ya son viables.

La integridad de los nuevos calendarios generados tras estas operaciones está garantizada por la primitiva `crear_juego`, la cual actúa como validador interno y corrige estructuras anómalas asignando valores aleatorios válidos ante cualquier incongruencia.

3.1.1.e. Escoger el método para evaluar los resultados y el criterio de terminación

El proceso de evaluación, selección, aplicación de operadores genéticos y reemplazo se repite durante varias generaciones.

- *Selección*: Se utilizará el método de selección en torneo o rueda de ruleta, donde los individuos con mejor aptitud tienen mayor probabilidad de ser elegidos.

- *Criterio de terminación*: El proceso se detendrá al alcanzar un número máximo de generaciones, propuesto en `NGEN` = 20. Este límite se determinó a partir de pruebas preliminares, donde se observó un estancamiento en la mejora de la función de aptitud. Por lo tanto, 20 iteraciones ofrecen un balance adecuado para la exploración del espacio de búsqueda considerando el rendimiento computacional.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/roza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

3.2. Arquitectura

En esta sección se describen los componentes principales, las funciones que realizan y la forma en que interactúan entre sí. El método implementado para la inicialización aleatoria se encuentra en el algoritmo 3.2 *crear_calendario_inicial()*.

Por otro lado, el método implementado para evaluar los resultados se encuentra en el algoritmo 3.3 *eval_calendario*(T). Aunque el programa evolutivo busca optimizar los resultados generación tras generación, su eficacia computacional final frente a los calendarios aleatorios iniciales será objeto de evaluación en el análisis de resultados del Capítulo 5.

Para validar la lógica de la función, se presentan tres escenarios típicos de evaluación (asumiendo que el número de juegos por equipo es igual a 2 para los tres equipos A, B y C). C_1 y C_2 representan elementos en el vector multiobjetivo:

El programa evolutivo requiere una población inicial a la cual aplicar las operaciones genéticas. Se propone un método para generar calendarios aleatorios como se muestra en el algoritmo 3.2.

Las operaciones genéticas se implementarán usando la librería DEAP, ya que tiene un soporte nativo para árboles de sintaxis. Los métodos disponibles se detallan en la documentación oficial. Para esta versión, se seleccionaron y adaptaron las funciones específicas: *genGrow* para la inicialización, *selTournament* para la selección, *cxOnePoint* para el cruzamiento y *mutNodeReplacement* para la mutación. La eficiencia de estas operaciones dependerá del volumen de los datos procesados. La

© Leslie Pamela Briones Valencia
Tesis IPN 2026 · github.com/lespam/koza-tga

Copia de preservación · Prohibida su reproducción sin citar la fuente

búsqueda del individuo más apto puede realizarse sobre la población total y de ser requerido sobre muestras aleatorias.

La infraestructura de la base de datos es creada mediante programación orientada a objetos para la creación de modelos con propiedades y métodos asociados a Django.

Los modelos a desarrollar para interactuar con la base de datos son: equipo, liga, torneo, calendario, serie, juego, calculadora y la configuración (del programa genético). Estas clases permiten crear y manipular objetos para interactuar con la base de datos mediante métodos heredados, los cuales incluyen implícitamente un identificador único.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Algoritmo 3.2: Método para la inicialización aleatoria

Nombre : *crear_ calendario_ inicial()*

Precondiciones: v_0 es el número total de juegos, p el número de juegos por serie, r el número de series, L_f una lista de fechas, L_E una lista de n equipos y n es par.

Entrada : Vacía

Salida : Un calendario T donde $C_1(T) = 0$.

```
juegos ← [ ]
lista_de_equipos ← [ ]
f ← 0;
// agrupa juegos por serie hasta agotar las fechas
for k ← 0 hasta r − 1 do
    mezclar_aleatoriamente(L_E);
    pares ← agrupar L_E de dos en dos;
    for i ← 0 hasta p − 1 do
        if f ≥ |L_f| then
            break;
        fecha ← L_f[f];
        for cada par ∈ pares do
            juego ← crear_juego(k, fecha, par[0], par[1]);
            juegos ← juegos ‖ juego;
regresar juegos;
```

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Algoritmo 3.3: Cálculo de la aptitud de un programa que representa un calendario

Nombre : *eval_ calendario*(T)

Precondiciones: v_0 es el número esperado de juegos, $L_E = (E_1, E_2, \ldots, E_n)$ es una lista de equipos que incluye las coordenadas de las sedes, y *calcula_ distancia* es un método definido por el usuario para calcular la distancia.

Entrada : Un programa calendario T (lista de juegos).

Salida : Vector de aptitud (C_1, C_2).

$suma_error \leftarrow 0$;

for *cada equipo* $E_i \in L_E$ **do**

  $v_i = contar(E_i, T)$;

  $suma_error \leftarrow suma_error + |v_0 - v_i|$;

$C_1 \leftarrow -suma_error$; `// Valor negativo para maximización`

$distancia_total \leftarrow 0$;

for *cada juego* $(f, E_A, E_B) \in T$ **do**

  $d = calcula_distancia(E_A, E_B)$;

  $distancia_total \leftarrow distancia_total + d$;

$C_2 \leftarrow distancia_total$;

regresar (C_1, C_2);

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 | github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Tabla 3.3. Ejemplos del cálculo de la aptitud

Caso	**Calendario (T)**	**Frecuencia de (A, B y C)**	C_1
Ideal	$[(f, A, B), (f, A, C), (f, B, C)]$	2, 2, 2	0
Incompleto	$[(f, A, B), (f, A, C)]$	2, 1, 1	-2
Desbalanceado	$[(f, A, B), (f, C, A), (f, A, C)]$	3, 1, 2	-2

C_2 corresponde a la sumatoria de distancias de cada par $d(A, B)$, $d(B, C)$ y $d(A, C)$.

Fuente: Elaboración propia.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/espanakoza-ttp

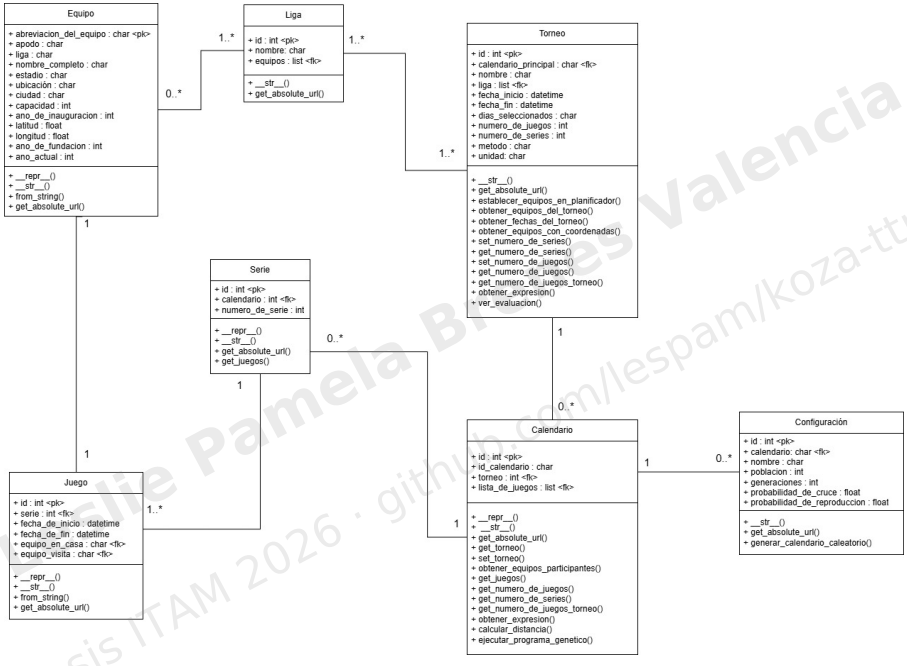
Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

3.2.1. Modelos para la base de datos

La estructura técnica de los modelos principales se encuentran en el diagrama UML de la figura 3.1. Las principales clases son:

Figura 3.1. Diagrama de clases



Se muestra la relación de objetos para la arquitectura de la base de datos.

- *Equipo*: representa a un equipo de béisbol.
- *Liga*: representa a una liga de béisbol.
- *Calendario*: representa un calendario de juegos.
- *Juego*: representa un juego individual de béisbol.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

- *Serie*: representa una serie de juegos.
- *Torneo*: representa los metadatos y configuraciones temporales de la competencia.
- *Configuración*: representa los parámetros para ejecutar el programa genético.

3.2.2. Módulos desarrollados

Se identificaron tres módulos lógicos a desarrollar de forma independiente, los cuales interactúan como se muestra en el diagramas de eventos de la figura 3.2. La estructura de los componente se encuentra en la figura 3.3.

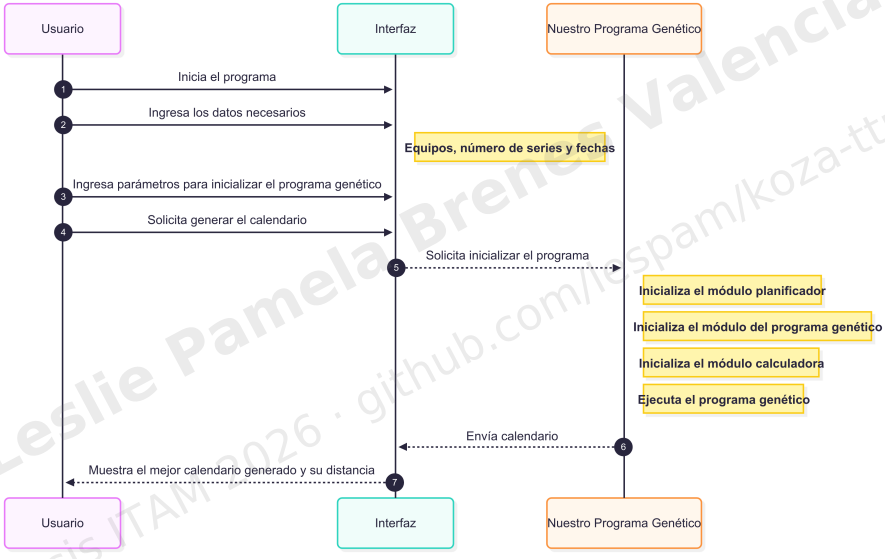
- *Módulo planificador.* Recibe y procesa la información de los equipos. Genera la representación del calendario y contiene la lista de equipos con coordenadas. Es el punto de comunicación central entre la interfaz web y el programa genético.
- *Módulo calculadora de distancia.* Componente independiente encargado de calcular la distancia entre dos puntos. Aislarlo permite cambiar y experimentar con distintas métricas (ej. euclídea o haversine) de forma transparente.
- *Módulo programa genético.* Inicializado por el planificador para generar calendarios aleatorios e iterar la búsqueda de optimización evaluando las poblaciones generadas.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozuttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 3.2. Diagrama de eventos



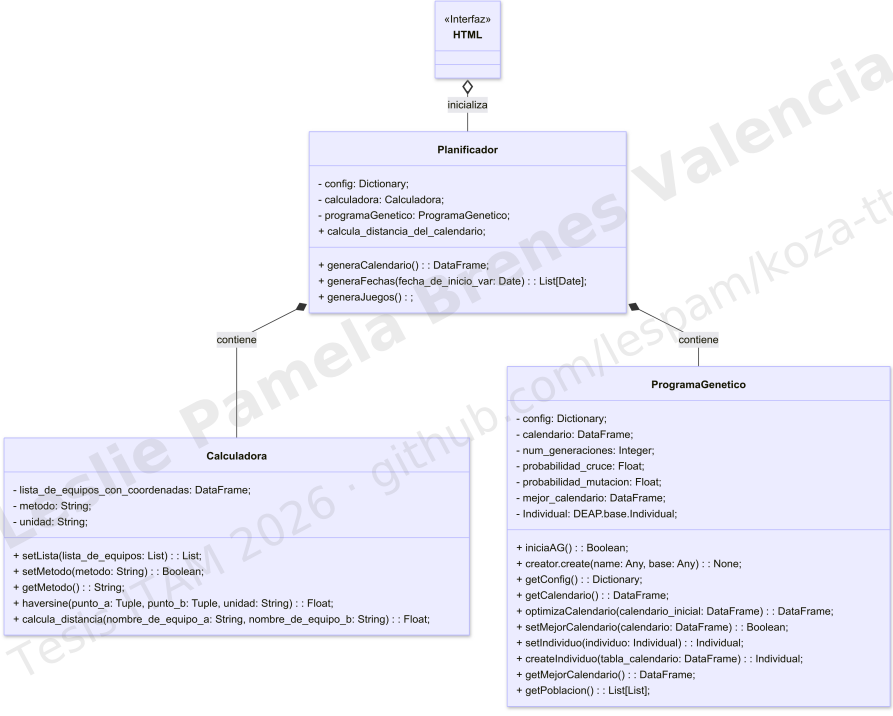
© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

El usuario interactúa en una sola aplicación que coordina la interoperabilidad de los componentes, los modelos, y sus valores en la base de datos. Fuente: Elaboración propia.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura 3.3. Diagrama de Interfaces



Módulos independientes que se comunican mediante una interfaz común. Fuente: Elaboración propia.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

3.3. Estándares utilizados

En esta sección se describen los principales estándares que se aplicarán en el proyecto. La unidad de distancia por defecto será el kilómetro (km), aunque también se aceptará la milla (mi) como alternativa. Las fechas se expresarán en el formato año-mes-día (`aaaa-mm-dd`). Para la navegación Web se usará el protocolo de transferencia de hipertexto (del inglés *Hypertext Transfer Protocol*) (HTTP), el lenguaje de marcado de hipertexto (del inglés *HyperText Markup Language*) (HTML) y el localizador de recursos uniforme (del inglés *Uniform Resource Locator*) (URL).

El lenguaje de programación seleccionado para el desarrollo del proyecto es Python. Los paquetes y el marco de desarrollo que usan Python son:

- **Python 3.7** como lenguaje de programación base.
- **Django 4.1** como marco de desarrollo web.
- **DEAP 1.4.1** se empleará para implementar las operaciones de programación genética.
- **haversine 2.8.0** se utilizará para calcular distancias esféricas entre 2 lugares.
- **numpy 1.26.0** y **pandas 2.1.1** se emplearán para realizar operaciones matemáticas y estructurar datos.
- **pygraphviz 1.9** para generar las gráficas visuales de los nodos.

© Leslie Pamela Brenes Valencia
Tesis TAM 2026 · github.com/lespam/koza-tap

Copia de preservación · Prohibida su reproducción sin citar la fuente

- **MVT** Modelo-Vista-Plantilla (*template*) para generar páginas HTML.
- **ORM** para generar los esquemas de la base de datos.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Capítulo 4

Implementación

© Leslie Pamela Brenes Valencia
Tesis ITCM 2026 · github.com/lespam/koza-ttp

Este capítulo describe la implementación de la interfaz de usuario con la herramienta de desarrollo web Django y se incluye una descripción técnica del código en Python. Finalmente, se describen los desafíos encontrados durante la implementación y las estrategias utilizadas para superarlos.

4.1. Aplicaciones

El funcionamiento del sistema se fundamenta en la interacción coordinada de sus componentes. En este proyecto, se define como componente a cualquier módulo o función cuya lógica interna puede ser modificada sin alterar la arquitectura global del programa. Por ejemplo, se permite la modificación de los métodos y unidades dentro del componente encargado del cálculo de distancias. En este prototipo se da prioridad al uso de la norma del semiverseno debido a su precisión en comparación con la distancia euclidiana.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Para mayor claridad en la lectura de este capítulo, es importante señalar que toda mención a rutas o archivos específicos (por ejemplo, `tesis/models.py`) hace referencia directa a la estructura del código fuente del proyecto. Dado que el objetivo de este documento es analizar la lógica de implementación y no transcribir el código en su totalidad, los archivos íntegros pueden ser consultados en el repositorio oficial, cuyos detalles de acceso se especifican más adelante en la sección 4.3 (Especificaciones de software y hardware)

De acuerdo con nuestra infraestructura, se desarrollaron tres aplicaciones que forman el proyecto. Las aplicaciones integradas son:

1. *Tesis*: Contiene los modelos, vistas y conexiones maestras.
2. *Programa genético*: Contiene el código para este programa evolutivo.
3. *MyApp*: Aplicación por omisión del proyecto.

```
INSTALLED_APPS = [
    "myapp",
    "tesis",
    "programa_genetico",
]
```

La lógica de optimización se separa en módulos independientes para garantizar la flexibilidad del sistema. En este caso los siguientes archivos contienen cada módulo implementado:

- El archivo `programa_genetico/calculadora.py` implementa la lógica de la métrica geográfica utilizando la librería `haversine` y su

Copia de preservación · Prohibida su reproducción sin citar la fuente

diseño modular facilita el reemplazo por otras métricas de ser necesario.

- El archivo `programa_genetico/planificador.py` contiene el *ModuloPlanificador()*, encargado de transformar los datos del torneo, generar el diccionario de juegos y proveer el método para crear calendarios iniciales viables ($C_1 = 0$).

- El archivo `programa_genetico/programagenetico.py` utiliza la biblioteca *DEAP* para implementar operaciones evolutivas prefabricadas y permite la manipulación de calendarios mediante árboles de sintaxis. No obstante, requiere la parametrización de diversos métodos y procesos evolutivos. Contiene la definición de la función de aptitud y la lógica del proceso evolutivo.

4.1.1. Programa genético

Para abordar el problema de validez en el número de juegos, se diseñó una función de evaluación multiobjetivo ya que el uso exclusivo de la distancia generaba calendarios inválidos. Se configuró el objeto `FitnessMin` de DEAP para maximizar este valor (buscando que tienda a cero) con un peso de 100.0, y minimizar la distancia con un peso de -1.0 (*weight = (100.0, -1.0)*). La figura 4.1 presenta la definición de las variables involucradas en el proceso de implementación del paradigma de la PG.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/espam/kozatto

Copia de preservación · Prohibida su reproducción sin citar la fuente

Tabla 4.1. Definición de variables de entrada y parámetros de salida

Variables requeridas	
L_E (Equipos)	Lista de equipos con coordenadas.
L_F (Fechas)	Lista de fechas disponibles.
v_o	Número de juegos esperado para cada equipo.
r	Número de series.
`calcula_distancia`	Método y unidad para calcular la distancia entre las sedes.
CXPB	Probabilidad de cruzamiento.
MUTPB	Probabilidad de mutación.
POP	Tamaño de la población.
NGEN	Número de generaciones.
Resultados	
T (Calendario)	Árbol de sintaxis con la programación de juegos optimizada.
C_1 (Diferencia)	Diferencia acumulada en la cantidad de juegos entre equipos.
C_2 (Distancia)	Suma total de distancias recorridas.
Tiempo (s)	Duración total del proceso en segundos.

Se muestran los parámetro requeridos para modelar un programa genético usando la librería DEAP. Se requiere la definición de los operadores genéticos y se debe proporcionar un método para inicializar aleatoriamente la primera generación de individuos. Fuente: Elaboració propia.

© Leslie Pamela Briones Valencia · Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

4.1.2. Vistas y Modelos

Se eligió el marco de desarrollo Django debido a su robusta arquitectura Modelo-Vista-Plantilla (del inglés *Model-View-Template*) (MVT) y a su mapeo objeto-relacional (del inglés *Object-Relational Mapping*) (ORM) integrado. Esta elección técnica agiliza la persistencia de datos y facilita la comunicación con la interfaz de usuario. La creación de modelos se realiza definiendo clases en Python que heredan de las clases con base en el diseño del ORM de Django.

Los objetos se representan mediante clases con atributos (por ejemplo, el nombre de un equipo), y con métodos inherentes a dicho objeto, permitiendo una interacción dinámica y estructurada antes de enviar la información a las vistas.

La arquitectura relacional y el diagrama de clases que se implementaron corresponden a los diagramas presentados previamente en la sección 3.2 (Arquitectura).

El archivo `tesis/models.py` orquesta la interacción entre la base de datos y el `ModuloPlanificador` para ejecutar el programa de forma dinámica.

Las vistas procesan las solicitudes HTTP y generan páginas de Internet. Se implementó la base de datos por defecto *db.sqlite3* mediante el motor *django.db.backends.sqlite3*.

© Lesly Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tdp

Copia de preservación · Prohibida su reproducción sin citar la fuente

4.1.3. Parámetros de la implementación

La figura 4.2 contiene los parámetros que se utilizaron durante esta implementación y que corresponde a los resultados.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Tabla 4.2. Resumen de la implementación

Modelo	**Origen**
Equipo, Liga, Torneo	Datos del usuario y estructura de la liga.
Calculadora	El usuario selecciona el método para la métrica y las unidades, usando el método del semiverseno y kilómetros por defecto.
Configuración	Parámetros dinámicos para el programa genético (POP, NGEN, CXPB, MUTPB).
Instancia del torneo	**Valores**
Nombre y liga	Torneo Prueba, MLB ficticia.
Periodo	20 de marzo al 2 de noviembre de 2024.
Días de juego	Lunes, martes, miércoles, viernes y sábado.
Total de juegos (v_0)	163 juegos programados.
Juegos por serie (p)	5 juegos por serie.
Número de series (r)	33 series.
Cálculo y medida de distancia	Método de semiverseno en kilómetros (km).
Parámetros del PG	**Configuración**
L_E	Lista de equipos asociados.
L_F	Lista de fechas filtradas.
POP	100 individuos.
NGEN	20 generaciones.
CXPB	0.8 para el proceso evolutivo y 0.0 para el proceso aleatorio.
MUTPB	0.2 para el proceso evolutivo y 0.0 para el proceso aleatorio.

Se muestra la definición de los parámetros. Durante la ejecución estos valores corresponden a los solicitados por el usuario y que se encuentran en la base de datos. Fuente: Elaboración propia.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/kona-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

4.2. Especificaciones de software y hardware

El código fuente del proyecto se encuentra alojado en un repositorio de GitHub disponible en https://github.com/lespam/koza-ttp. Actualmente puede encontrarse una demostración funcional de la aplicación en el sitio: https://koza-ttp.lesliedickinson.dev.

Las pruebas de ejecución se realizaron en un equipo con las siguientes especificaciones:

- **Sistema Operativo:** Windows 11 de 64 bits.
- **Memoria RAM:** 32 GB.
- **Procesador:** 8 núcleos a 3.7 GHz.
- **Entorno:** Python 3.7 y Django 4.1.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Capítulo 5

Resultados

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

La implementación de una aplicación web con Django permitió integrar en una sola interfaz la comunicación entre componentes, la generación de pruebas para su análisis y la visualización de las soluciones. Este capítulo describe esas pruebas y los resultados obtenidos.

5.1. Descripción de las pruebas

Durante la implementación del programa genético, se realizaron diversas pruebas para evaluar su efectividad en la generación de calendarios deportivos.

En primer lugar, se verificó que el programa fuera capaz de generar distintas soluciones a petición del usuario. En segundo lugar, que el programa fuera capaz de generar correctamente el número de juegos requerido. Finalmente, se comparó la ejecución del algoritmo con y sin la intervención de operadores genéticos durante la implementación del

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

paradigma de la PG. Cuando la probabilidad de cruce y mutación es cero, el algoritmo actúa simplemente como un seleccionador de calendarios generados aleatoriamente.

De este modo, la evaluación se estructuró en dos escenarios. La primera, una prueba de concepto con calendarios aleatorios. Para esta iteración se simplificó el diseño al asignar un juego por día, un juego por los días de la semana de juego equivalente a una serie (aunque en la realidad de las ligas podrían existir juegos dobles y de más estructuras). Establecido este punto de referencia, el segundo objetivo fue aplicar el método de optimización evolutiva para evaluar si la inclusión de operadores genéticos lograba superar el óptimo local generado aleatoriamente.

Dada la complejidad de la implementación, solo se mostrarán los resultados de una ejecución por caso. Sin embargo, cabe reiterar que este es un prototipo, por lo que las pruebas verificaron primordialmente el dinamismo del programa.

Al final del capítulo se profundizará en los aprendizajes y retos enfrentados y se sugiere continuar experimentando con múltiples ejecuciones.

El primer prototipo del programa producía calendarios que no cumplían con el número de juegos requerido para cada equipo. El algoritmo concentrado en minimizar la distancia resultaba en calendarios incompletos y la cantidad de juegos era distinta entre los diferentes equipos. Para abordar este problema, se decidió modificar el diseño y replantear el criterio de aptitud del programa.

Se optó por darle prioridad a la generación de calendarios con una

Copia de preservación · Prohibida su reproducción sin citar la fuente

cantidad de juegos constante para todos los equipos. Para lograrlo, se calculó un parámetro de error entre el número de juegos observado contra el esperado por equipo. Minimizar la distancia total pasó a ser la segunda prioridad del programa. El diseño ahora tenía dos objetivos y el módulo DEAP es capaz de optimizar múltiples objetivos de forma simultánea. Estos se representan mediante un vector donde cada elemento es un número real que representa el peso de esa variable. La clase *Fitness* contiene el parámetro de peso (*weight*). No existe una regla fija para definir qué valores usar, dependen de las necesidades del problema.

A este nuevo parámetro se le asignó una prioridad de 100 contra 1 para dar preferencia a los calendarios con el mismo número de juegos. Mientras tanto, a la aptitud de distancia se le asignó el valor de -1 para indicar que se busca maximizar el primer parámetro y minimizar el segundo. Ejemplo: $weight = (100, -1)$.

Desafortunadamente, en muchos casos el algoritmo se quedaba con el primer calendario generado aleatoriamente, ya que este cumplía con el número de juegos requerido, dejando en segundo plano la minimización de la distancia. Esto limitaba la efectividad del programa, ya que no exploraba otras soluciones que podrían haber sido mejores en términos de distancia total.

Por último, las infinitas posibilidades de configurar los operadores evolutivos brindó infinitos espacios de soluciones posibles por lo que se decidió primero registrar estos resultados. Una vez que el prototipo se consideró funcional, se decidió registrar las generaciones igual a 20, la población igual a 100 y se validó el dinamismo del programa ante cambios

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lepam/kozu-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

en el número de juegos y equipos.

5.2. Análisis de resultados

Entender el funcionamiento de los procesos de cruce y mutación en árboles de sintaxis influye en las soluciones en la programación genética.

Durante la fase de experimentación, se realizaron diversos ajustes y se logró el diseño de primitivas y terminales de acuerdo con el paradigma de la programación genética, y se procedió a validar la prueba de concepto.

Sin embargo, es importante recordar que los operadores prefabricados modifican nodos y ramas en un árbol de sintaxis pero no tienen en cuenta el contexto del problema específico.

La tabla 5.1 señala la descripción de los parámetros de entrada y salida usados. Se compararon calendarios con y sin la aplicación de operadores genéticos para un ejemplo donde se tienen los 30 equipos de la MLB, la lista de sedes con sus coordenadas, un número de 163 juegos y series de 5 juegos a la semana.

Durante la ejecución, se observó que la generación inicial ya incluía el mejor calendario que el programa entregó al final del proceso. Esto quedó evidenciado en la primera generación, donde todos los calendarios cumplían con el requisito del número de juegos.

© Leslie Pamela Briones Valencia
Tesis MAM 2026 github.com/lespam/mozart-ao

Copia de preservación · Prohibida su reproducción sin citar la fuente

Tabla 5.1. Parámetros de entrada y salida

Parámetros del experimento	
Parámetro	**Descripción**
CXPB	Probabilidad de cruce, determina la frecuencia de recombinación de individuos.
MUTPB	Probabilidad de mutación, determina cambios aleatorios en los individuos.
NGEN	Número de generaciones para el criterio de terminación.
POP	Tamaño de la población inicial.
Resultados del experimento	
Diferencial de juegos	Diferencia en la cantidad de juegos entre equipos en el mejor calendario generado.
Distancia	Distancia total recorrida en el calendario optimizado.
Tiempo (s)	Tiempo total de ejecución del algoritmo en segundos.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/despam/koza-ttp

En un futuro se podrían modificar estos métodos, pero se priorizó entender la implementación completa. Con base en este entendimiento, se sugiere un futuro análisis del problema por partes:

- *Ajuste fino de parámetros*: Realizar un ajuste sobre las probabilidades de cruce, mutación, tamaño de población y número de generaciones, ya que estos influyen drásticamente en la capacidad del algoritmo. Por lo tanto, es importante considerar que los valores dependerán del problema, y será necesario perfeccionar la ponderación de los objetivos de aptitud. Todo esto tiene efectos en el espacio de búsqueda.

Copia de preservación · Prohibida su reproducción sin citar la fuente

- *Desarrollo de operadores específicos:* Los operadores de cruce y mutación genéricos utilizados no están intrínsecamente adaptados al problema de programación de calendarios. Se requiere adaptar el método de intercambio de árboles de sintaxis para manipular estructuras en el estricto contexto de la representación de juegos y no como simples nodos abstractos.

- *Refinamiento del proceso de búsqueda*: Cambiar el enfoque del algoritmo para que, en lugar de generar calendarios desde cero, perfeccione soluciones iniciales. El programa, en lugar de ser un método que genera calendarios desde cero (por ejemplo, `genera_calendario()`), se podría crear un método que permita la búsqueda a partir de soluciones ya válidas o prometedoras (por ejemplo, `genera_calendario(datos_iniciales)`).

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-tap

5.3. Generador de calendarios sin la PG

Tabla 5.2. Parámetros y resultados del experimento de optimización genética

Parámetros	
CXPB	0
MUTPB	0

Resultados	
Dif. de juegos	0
Distancia	2,663,470.70 km
Tiempo	200.25 s

Copia de preservación · Prohibida su reproducción sin citar la fuente

Cuando no se aplica el paradigma de la PG (de acuerdo a la figura 5.2), los resultados muestran el proceso de selección aleatoria. Esta selección es ejecutada mediante el método `toolbox.selTournament` proporcionado por DEAP. Se selecciona el mejor individuo entre una lista de los mejores k ($k = 100$) individuos aleatoriamente seleccionados, con reemplazo.

Las demás generaciones no requieren de ninguna evaluación porque ningún individuo pasa por el proceso de cruzamiento y mutación. Finalmente, para la tercera generación se seleccionó siempre el mismo calendario y la desviación estándar es de 0. Este fue el programa que entregó el mejor resultado. La tabla de resultados se encuentra en la figura 5.3.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/kozattp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Tabla 5.3. Evaluación de valores de aptitud por generación (Búsqueda Aleatoria)

Generación		Juegos incompletos				Aptitud			
Gen	Evals	Desv. Est.	Mín.	Prom.	Máx.	Desv. Est.	Mín.	Prom.	Máx.
0	100	0	0	0	0	648591	2.66347e+06	4.90828e+06	6.24857e+06
1	0	0	0	0	0	412288	2.66347e+06	3.76291e+06	4.49158e+06
2	0	0	0	0	0	393205	2.66347e+06	2.99754e+06	3.56433e+06
3	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
4	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
5	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
6	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
7	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
8	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
9	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
10	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
11	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
12	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
13	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
14	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
15	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
16	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
17	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
18	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06
19	0	0	0	0	0	0	2.66347e+06	2.66347e+06	2.66347e+06

© Leslie Pamela Brenes Valverde
Tesis ITAM 2026 · github.com/lesbam/kobe-tfg

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

5.4. Generador de calendarios con la PG

Tabla 5.4. Parámetros y resultados del experimento de optimización genética

Parámetros	
CXPB	0.8
MUTPB	0.4

Resultados	
Dif. de juegos	0
Distancia	3,182,738.33 km
Tiempo	3,337.81 s

Se experimentó con distintos parámetros y aquí solo se muestra una ejecución (ver la figura 5.4). La optimización de estos parámetros mediante un barrido sistemático se consideró fuera del alcance de los objetivos de este prototipo.

Los resultados demuestran que el proceso es evolutivo, dado que cada generación es distinta. Sin embargo, el calendario finalmente seleccionado no fue resultado de la PG.

La explicación se encuentra en el método de selección del mejor individuo. Se usó la función `tools.HallOfFame(1)` para conservar el mejor individuo absoluto. Como consecuencia, haber usado el cruzamiento genérico `gp.cxOnePointLeafBiased` y la mutación `gp.mutNodeReplacement`, con los parámetros reportados, no fue suficiente para romper los óptimos locales y generar soluciones superiores a las generadas al azar. Los resultados se encuentran en la figura 5.5.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lcpam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Tabla 5.5. Resultados de aptitud y juegos incompletos a lo largo de las generaciones (PG)

Generación		Juegos incompletos				Aptitud			
Gen	Evals	Desv. Est.	Mín.	Prom.	Máx.	Desv. Est.	Mín.	Prom.	Máx.
0	100	0	0	0	0	528327	3.18274e+06	5.07258e+06	5.96221e+06
1	85	1017.4	-3560	-1084.04	0	1.30114e+06	1.05372e+06	4.06517e+06	7.38415e+06
2	89	1080.15	-4016	-1230.42	0	1.29245e+06	695275	3.69353e+06	7.21956e+06
3	92	1231.6	-4294	-1316.92	0	1.29835e+06	473089	3.46033e+06	6.59003e+06
4	90	1156.78	-3898	-1264.48	0	1.24622e+06	646513	3.36871e+06	6.92115e+06
5	88	992.826	-3696	-914.84	0	924016	786403	3.23208e+06	6.02501e+06
6	89	1107.57	-4110	-1087	0	1.02072e+06	507676	3.19749e+06	5.8578e+06
7	91	1037.88	-4292	-1265.82	0	1.06844e+06	389220	3.2027e+06	5.97626e+06
8	85	1177.26	-4046	-1315.54	0	1.14911e+06	548805	3.17993e+06	5.81712e+06
9	90	1216.56	-4276	-1175.74	0	1.101e+06	401660	3.18407e+06	5.96382e+06
10	93	1155.67	-3614	-1351.76	0	1.15723e+06	831194	3.18261e+06	5.53428e+06
11	88	1318.41	-4724	-1290.1	0	1.20054e+06	106851	3.181e+06	6.25558e+06
12	89	1282.06	-4066	-1403.9	0	1.23722e+06	539332	3.18063e+06	5.71067e+06
13	83	1347.24	-4022	-1380.28	0	1.2551e+06	564954	3.18262e+06	5.80052e+06
14	91	1234.22	-3998	-1390.58	0	1.21018e+06	579967	3.18246e+06	5.78421e+06
15	93	1035.56	-4644	-1123.3	0	994200	158492	3.18666e+06	6.20698e+06
16	88	1235.64	-4484	-1491.12	0	1.26042e+06	264392	3.18192e+06	6.07259e+06
17	92	1230.89	-4154	-1260.02	0	1.1464e+06	479781	3.18057e+06	5.88354e+06
18	92	1149.03	-4040	-1370.42	0	1.16348e+06	554884	3.17153e+06	5.80844e+06
19	89	1408.58	-5184	-1108.6	0	1.16669e+06	97630	3.18717e+06	6.55169e+06

© Leslie Pamela Briones Valencia
Tesis ITAM 2025 · github.com/lespam/tesis-itam

5.5. Desafíos y aprendizajes

El resultado fundamental es que se implementó de forma exitosa un prototipo funcional que aborda la problemática de la planificación de temporadas de béisbol, cumpliendo así con el objetivo específico:

- El sistema desarrollado es capaz de generar y evaluar calendarios con distintos equipos, sedes y número de juegos.

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

- En el ejemplo presentado, se logró entregar un calendario cuyo óptimo local encontrado tuvo una distancia total de 2,663,470.70 km y un diferencial de juegos de 0.

Si bien demostró ser funcional a nivel de arquitectura, no logró una mejora significativa mediante la adaptación del paradigma de la PG en esta etapa.

Los resultados son positivos en términos de ingeniería de software, demostrando la viabilidad y dinamismo de la programación evolutiva para modelar la complejidad de generar horarios deportivos.

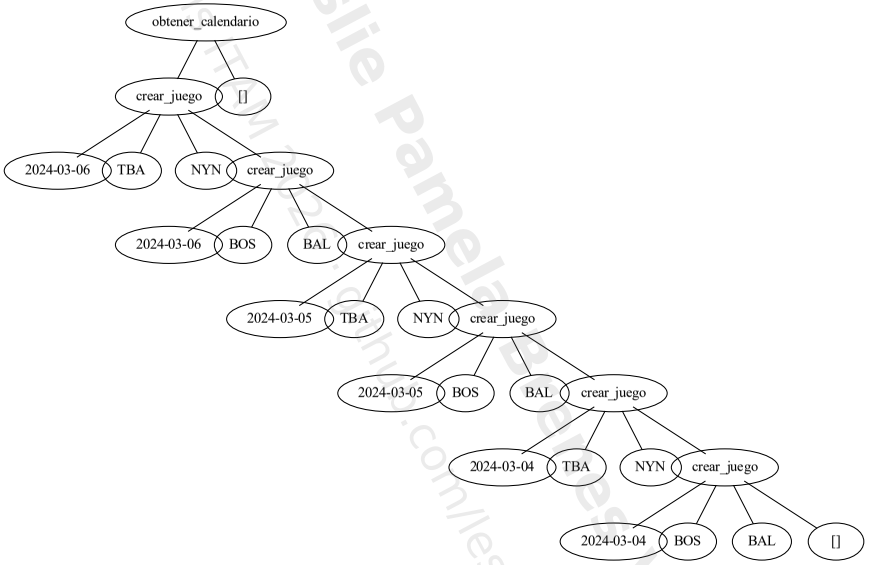
En este trabajo, uno de los mayores retos fue entender estos conceptos e integrarlos en un programa que cumpliera con el diseño y propósito establecidos. La integración de conceptos teóricos en una aplicación práctica requirió una adaptación detallada de los mecanismos de la programación evolutiva. La elección del paradigma se fundamentó en la naturaleza jerárquica de los calendarios, la cual es compatible con la estructura de representación de la programación genética. El diseño ingenieril fortaleció las decisiones que se tomaron al realizar un análisis del problema y de las posibles soluciones, y se desafiaron intuiciones iniciales.

Una vez superado el reto creativo, se continuó con la implementación de una interfaz web. El principal desafío técnico de la implementación consistió en el proceso de serialización e interpretación de datos entre listas, cadenas de texto y árboles de sintaxis, como se ilustra en la figura 5.1

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lesplin/kuza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Figura 5.1. Calendario



Árbol de sintaxis que representa un calendario. En este caso inicia el proceso con una lista vacía y un juego aleatorio. Luego, se agregan juegos mediante la función *crear_juego*. Finalmente, el usuario recupera la lista completa mediante la función *obtener_calendario*. Gráfica elaborada usando la librería DEAP.

© Leslie Pamela Pérez Valencia
Tesis en Maestría en Computación
Copia de preservación · Prohibida su reproducción sin citar la fuente
Versión pre-oficial

Versión pre-oficial

Capítulo 6

Conclusiones

El desarrollo ingenieril fue exitoso en resolver el problema inicial de planificación de calendarios. Su resultado fue una interfaz web dinámica capaz de generar y gestionar calendarios de torneos de béisbol genéricos. El prototipo generado cumplió el objetivo de crear una prueba de concepto, demostrando que es posible utilizar PG para generar calendarios deportivos. Por una parte, se cumple con la restricción donde todo equipo debe participar en el número de juegos estipulado por el torneo y, adicionalmente, se optimiza la distancia total de traslado. Sin embargo, las soluciones presentadas aún están lejos de ser aplicables en escenarios reales, donde se deben considerar restricciones y formatos adicionales.

Por lo tanto, el sistema desarrollado es capaz de generar y evaluar calendarios con distintos equipos, sedes y número de juegos. Inesperadamente, la solución final siempre fue la generada por el proceso de inicialización aleatoria. Aunque se logró transformar la información en

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

una estructura jerárquica apropiada para la PG, el calendario óptimo nunca fue procesado inteligentemente por el programa genético, debido a que el individuo inicial cumplía las restricciones de viabilidad y el algoritmo convergió prematuramente en un óptimo local. Esto limitó la efectividad evolutiva del programa, pues no se exploraron ramas que aprovecharan la información de calendarios generados previamente a través de operadores de cruce y mutación más específicos.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Epílogo

El futuro de la IA y la programación genética

La inteligencia artificial ha adquirido una relevancia significativa en diversos ámbitos de la vida cotidiana durante los últimos años. Sin embargo, es común confundir una tecnología avanzada con una tecnología verdaderamente inteligente. La programación genética es un ejemplo de cómo podemos utilizar conceptos biológicos para resolver problemas complejos mediante la evolución de estructuras de datos.

Como menciona Kuri (2000, p. 21), *en optimización, al igual que en muchas otras situaciones de la vida cotidiana, no existe* el método. *La conveniencia de cada uno depende del problema que se pretende resolver.* En este trabajo se exploró una de las infinitas formas de modelar un problema.

Es posible que la computación cuántica permita un escenario donde los parámetros de evolución se adapten dinámicamente a la naturaleza del problema. Esta tecnología emergente tiene el potencial de ejecutar procesos en paralelo con múltiples estados iniciales gracias al uso de q-bits.

© Leslie Fermula Prenes Valencia
Tesis ITAM 2026 · github.com/lesprenes/k-za-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

El uso de q-bits podría revolucionar la programación evolutiva obteniendo información del proceso evolutivo mediante un barrido de diversas configuraciones simultáneas, abriendo la puerta a la meta-programación evolutiva. Hoy, tanto la inteligencia artificial como la natural coexisten y contribuyen al desarrollo de nuestra propia evolución.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Apéndices

I. Análisis de distintas ligas profesionales de béisbol

Además del torneo de la MLB, se analizaron cuatro ligas de tres países para identificar los parámetros comunes requeridos en la estructuración de una liga genérica. Esta diversidad geográfica y logística valida la necesidad de una infraestructura de software flexible, como la desarrollada en este proyecto.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

I.1. Liga Mexicana de Béisbol (LMB)

En el caso de la Liga Mexicana de Béisbol (LMB), existen dos divisiones: zona norte y zona sur. Cada división se compone de nueve equipos. Se revisó la temporada 2022, la cual se compuso de 90 juegos entre abril y agosto.

El mapa de la figura I.1 muestra las ubicaciones de los estadios al interior de México (Wikipedia Contributors, 2022a). El formato del calendario muestra los juegos por serie. Los días de serie son los martes, miércoles y jueves; y viernes, sábado y domingo, con dos series por

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

semana. Sin embargo, algunos juegos se realizan en otras fechas, usualmente con un día de diferencia (Azteca Deportes, 2022). Todos los equipos jugaron de acuerdo con el calendario mostrado en la figura I.2.

Figura I.1. Mapa de la Liga Mexicana de Béisbol



Fuente: Wikipedia Contributors, 2022a

© Leslie Pamela Cuenes Valencia · github.com/espam/koza-ttp · Tesis ITAM 2026

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura I.2. Calendario de la temporada 2022 de la LMB



Fuente: Azteca Deportes, 2022

I.2. Liga Mexicana del Pacífico (LMP)

En la Liga Mexicana del Pacífico (LMP), los diez equipos se enfrentan en 68 juegos divididos en dos vueltas. Las series son de tres partidos y cada serie se juega íntegramente durante la misma semana (Duarte, 2019). Al finalizar la primera vuelta, se genera el primer listado de la puntuación de la liga. Se juega durante el invierno en la región noroeste de México, como se puede ver en la figura I.3. En el calendario de la figura I.4 se identifica la transición entre la primera vuelta y la segunda vuelta

Copia de preservación · Prohibida su reproducción sin citar la fuente

(Wikipedia Contributors, 2022b), (Valverde, 2019).

Figura I.3. Mapa de la Liga Mexicana del Pacífico



Fuente: Wikipedia Contributors, 2022b.

© Leslie Pamela Bretas Valencia
Tesis ITAM 2026 · github.com/lespablk/kozattp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura I.4. Calendario de la Liga Mexicana del Pacífico 2019



Fuente: Valverde, 2019

I.3. Liga Venezolana de Béisbol Profesional (LVBP)

La Liga Venezolana de Béisbol Profesional (LVBP) se compone de ocho equipos que juegan 49 veces cada uno, para un total de 196 juegos en la

Copia de preservación · Prohibida su reproducción sin citar la fuente

temporada regular. Cada equipo debe disputar al menos siete juegos con todos los demás integrantes. El mapa de la figura I.5 muestra la ubicación de los equipos en Venezuela. El calendario se publica semanalmente, tal como lo ilustra la figura I.6 (Baseball-reference, s.f.).

Figura I.5. Mapa de la Liga Venezolana de Béisbol Profesional



Fuente: Wikipedia Contributors, 2022c.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/[redacted]/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura I.6. Calendario semanal para la Liga Venezolana de Béisbol Profesional



Semana del 29-11-2021 al 05-12-2021 | Temporada Regular

Hora (ET)	Visitante	Local	Resultado	Estatus	Canal
			29/11		
06:00 PM	Navegantes	Leones	MAG 16, CAR 3	Finalizado	STV/TLT/BYM Inter
			30/11		
06:00 PM	Caribes	Bravos	ANZ 9, MAR 3	Finalizado	IVC
06:00 PM	Cardenales	Aguilas	LAR 3, ZUL 2	Finalizado	BYM Inter
06:07 PM	Tiburones	Tigres	LAG 5, ARA 3	Finalizado	STV
			01/12		
06:00 PM	Caribes	Tigres	ANZ 10, ARA 0	Finalizado	IVC
06:00 PM	Cardenales	Aguilas	LAR 5, ZUL 3	Finalizado	STV
06:00 PM	Bravos	Leones	MAR 6, CAR 10	Finalizado	BYM Inter
			02/12		
06:00 PM	Bravos	Tigres	MAR 4, ARA 7	Finalizado	BYM Inter
06:00 PM	Leones	Tiburones	CAR 9, LAG 4	Finalizado	IVC
06:00 PM	Caribes	Navegantes	ANZ 14, MAG 8	Finalizado	TLT
06:07 PM	Cardenales	Aguilas	LAR 11, ZUL 5	Finalizado	STV
			03/12		
06:00 PM (DJ)	Navegantes	Cardenales	MAG 2, LAR 0	Finalizado	IVC
06:00 PM	Caribes	Tigres	ANZ 3, ARA 4	Finalizado	BYM Inter
06:00 PM	Tiburones	Leones	LAG 0, CAR 1	Finalizado	TLT
06:07 PM	Bravos	Aguilas	MAR 7, ZUL 4	Finalizado	STV
09:00 PM (DJ)	Navegantes	Cardenales	MAG 2, LAR 7	Finalizado	IVC
			04/12		
03:07 PM	Bravos	Aguilas	MAR 9, ZUL 8	Finalizado	STV
04:00 PM (DJ)	Caribes	Tiburones	ANZ 11, LAG 8	Finalizado	IVC
05:00 PM	Tigres	Cardenales	ARA 8, LAR 14	Finalizado	BYM Inter
06:00 PM	Leones	Navegantes	CAR 0, MAG 1	Finalizado	TLT
07:00 PM (DJ)	Caribes	Tiburones	ANZ 4, LAG 6	Finalizado	IVC
			05/12		
01:00 PM	Caribes	Leones	ANZ 9, CAR 14	Finalizado	IVC
02:30 PM	Bravos	Aguilas	MAR 2, ZUL 3	Finalizado	BYM Inter
05:07 PM	Cardenales	Tigres	LAR 6, ARA 5	Finalizado	STV
05:30 PM	Tiburones	Navegantes	LAG 4, MAG 12	Finalizado	TLT

(DJ) Doble Juego

Fuente: Liga Venezolana de Béisbol Profesional (2021)

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

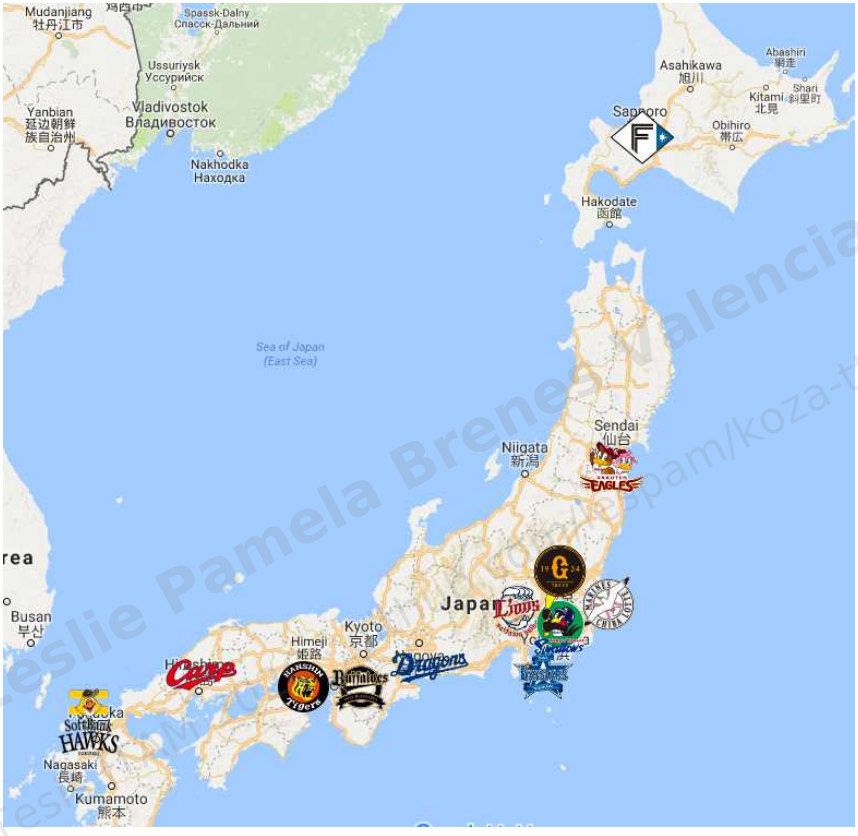
I.4. Nippon Professional Baseball league (NPB)

La Organización de béisbol profesional de Japón (del inglés *Nippon Professional Baseball league*) (NPB) cuenta con dos ligas principales: la Liga Central y la Liga del Pacífico. Cada liga tiene seis equipos, sumando un total de doce. En la figura I.7 se encuentra el mapa de la liga. Cada equipo juega 144 partidos por temporada, la cual usualmente se desarrolla de marzo a octubre. El calendario para la temporada 2019 se encuentra en la figura I.8. Este formato muestra los juegos por mes e identifica a los equipos por sus iniciales; sin embargo, no ofrece una visualización clara de series o divisiones.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Figura I.7. Mapa de la liga de béisbol profesional de Japón

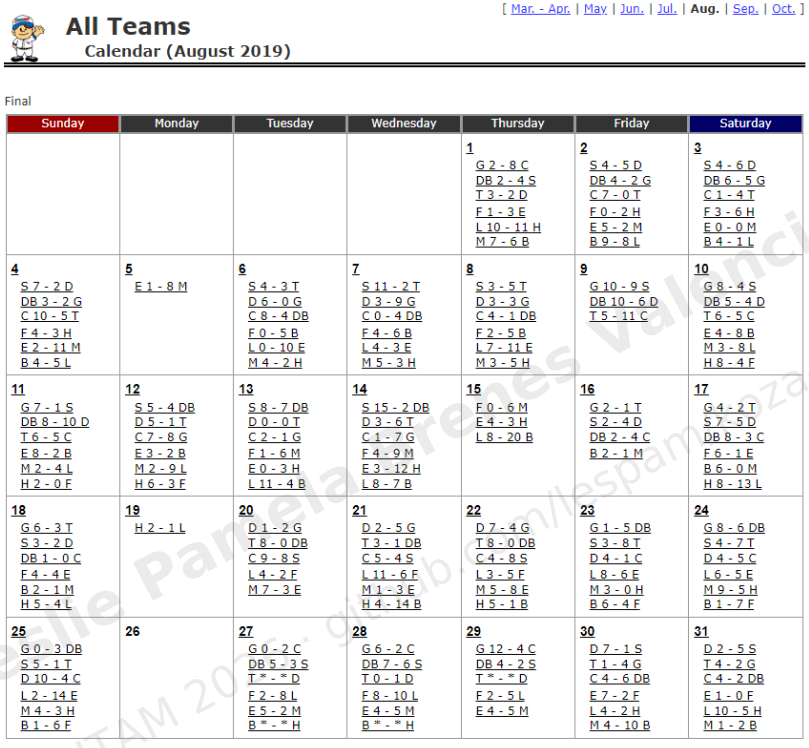


Fuente: Sport League Maps (2021)

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura I.8. Calendario de la liga de béisbol profesional de Japón. Agosto 2019



[Mar. - Apr. | May | Jun. | Jul. | **Aug.** | Sep. | Oct.]

All Teams
Calendar (August 2019)

Final

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
				1 G 2 - 8 C DB 2 - 4 S T 3 - 2 D F 1 - 3 E L 10 - 11 H M 7 - 6 B	**2** S 4 - 5 D DB 4 - 2 G C 7 - 0 T F 0 - 2 H E 5 - 2 M B 9 - 8 L	**3** S 4 - 6 D DB 6 - 5 G C 1 - 4 T F 3 - 6 H E 0 - 0 M B 4 - 1 L
4 S 7 - 2 D DB 3 - 2 G C 10 - 5 T F 4 - 3 H E 2 - 11 M B 4 - 5 L	**5** E 1 - 8 M	**6** S 4 - 3 T D 6 - 0 G C 8 - 4 DB F 0 - 5 B L 0 - 10 E M 4 - 2 H	**7** S 11 - 2 T D 3 - 9 G C 0 - 4 DB F 4 - 6 B L 4 - 3 E M 5 - 3 H	**8** S 3 - 5 T D 3 - 3 G C 4 - 1 DB F 2 - 5 B L 7 - 11 E M 3 - 5 H	**9** G 10 - 9 S DB 10 - 6 D T 5 - 11 C	**10** G 8 - 4 S DB 5 - 4 D T 6 - 5 C E 4 - 8 B M 3 - 8 L H 8 - 4 F
11 G 7 - 1 S DB 8 - 10 D T 6 - 5 C E 8 - 2 B M 2 - 4 L H 2 - 0 F	**12** S 5 - 4 DB D 5 - 1 T C 7 - 8 G E 3 - 2 B M 2 - 9 L H 6 - 3 F	**13** S 8 - 7 DB D 0 - 0 T C 2 - 1 G F 1 - 6 M E 0 - 3 H L 11 - 4 B	**14** S 15 - 2 DB D 3 - 6 T C 1 - 7 G F 4 - 9 M E 3 - 12 H L 8 - 7 B	**15** F 0 - 6 M E 4 - 3 H L 8 - 20 B	**16** G 2 - 1 T S 2 - 4 D DB 2 - 4 C B 2 - 1 M	**17** G 4 - 2 T S 7 - 5 D DB 8 - 3 C F 6 - 1 E B 6 - 0 M H 8 - 13 L
18 G 6 - 3 T S 3 - 2 D DB 1 - 0 C F 4 - 4 E B 2 - 1 M H 5 - 4 L	**19** H 2 - 1 L	**20** D 1 - 2 G T 8 - 0 DB C 9 - 8 S L 4 - 2 F M 7 - 3 E	**21** D 2 - 5 G T 3 - 1 DB C 5 - 4 S L 11 - 6 F M 1 - 3 E H 4 - 14 B	**22** D 7 - 4 G T 8 - 0 DB C 4 - 8 S L 3 - 5 F M 5 - 8 E H 5 - 1 B	**23** G 1 - 5 DB S 3 - 8 T D 4 - 1 C L 8 - 6 E M 3 - 0 H B 6 - 4 F	**24** G 8 - 6 DB S 4 - 7 T D 4 - 5 C L 6 - 5 E M 9 - 5 H B 1 - 7 F
25 G 0 - 3 DB S 5 - 1 T D 10 - 4 C L 2 - 14 E M 4 - 3 H B 1 - 6 F	**26**	**27** G 0 - 2 C DB 5 - 3 S T * - * D F 2 - 8 L E 5 - 2 M B * - * H	**28** G 6 - 2 C DB 7 - 6 S T 0 - 1 D F 8 - 10 L E 4 - 5 M B * - * H	**29** G 12 - 4 C DB 4 - 2 S T * - * D F 2 - 5 L E 4 - 5 M	**30** D 7 - 1 S T 1 - 4 G C 4 - 6 DB E 7 - 2 F L 4 - 2 H M 4 - 10 B	**31** D 2 - 5 S T 4 - 2 G C 4 - 2 DB E 1 - 0 F L 10 - 5 H M 1 - 2 B

© Leslie Pamela Arenes Valencia
Tesis ITAM 2020 - github.com/lespam/za-ttp

Fuente: Nippon Professional Baseball Organization (2019)

II. Análisis de modificaciones en el calendario de la MLB

Los cambios históricos en la MLB evidencian la necesidad de un programa para generar calendarios que sea rápido, flexible y robusto para adaptarse

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

a estos cambios estructurales.

II.1. Modificaciones de la liga

- En 1977, se agregaron dos equipos nuevos (Marineros de Seattle y Azulejos de Toronto), lo que provocó la reorganización de las divisiones.
- En 1993, la incorporación de los Rockies de Colorado y los Marlines de Miami permitió conformar tres divisiones por liga.
- En 1998, se integraron las Mantarrayas de Tampa Bay y los Cascabeles de Arizona.
- En 2004, los Expos de Montreal se convirtieron en los Nacionales de Washington D.C.
- En 2013, los Astros de Houston pasaron de la Liga Nacional a la Liga Americana.

II.2. Huelgas de jugadores y eventos externos

- En 1981, se redujeron los juegos a 103 hasta 111 por equipo debido a una huelga.
- En 1994, se cancelaron 713 juegos y la postemporada completa por una huelga.
- En 1995, la temporada se acortó a 144 juegos por el inicio retrasado derivado de la huelga anterior.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2020 · github.com/lespam/kozattp

Copia de preservación · Prohibida su reproducción sin citar la fuente

- En 2020, la pandemia de COVID-19 redujo el calendario a tan solo 60 juegos.

II.3. Cambios en las reglas y formato

- En 2012, se introdujo el juego de comodín, afectando el conteo de la temporada regular.
- En 2022, se incrementó la participación postemporada a 12 equipos y se reemplazó el comodín por una serie al mejor de tres.
- En 2023, se implementó el calendario balanceado donde todos los equipos se enfrentan entre sí, aumentando significativamente los juegos interligas.

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozattp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

III. Variación en el número de juegos de la MLB

La tabla III.1 detalla la fluctuación del número de juegos por temporada desde 1974, reflejando la flexibilidad necesaria en cualquier sistema de planificación deportiva.

Tabla III.1. Relación de juegos por temporada en la MLB (1974-2023)

Juegos	Año
60	2020
111	1981
117	1994
145	1995
162	1974, 1975, 1976, 1977, 1979, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991, 1992, 1993, 1996, 1997, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2010, 2011, 2012, 2014, 2015, 2016, 2017, 2019, 2021, 2022, 2023
163	1978, 1980, 1998, 1999, 2007, 2008, 2009, 2013, 2018

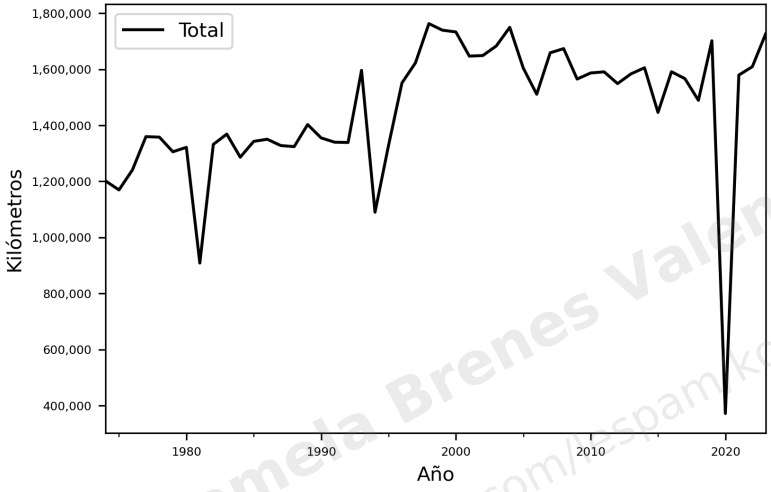
Aunque la mayoría de los torneos de la MLB constan de 162 juegos, esta cantidad es variable y se ajusta según las necesidades de la temporada. Datos de Sports Reference (2023).

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 github.com/lespam/koza-tfp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Figura III.9. Gráfica de la distancia total por año de la MLB



Variación de la distancia total recorrida anualmente, reflejando la estructura específica de cada torneo. Datos de Baseball Savant (2023)

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/espamkoza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Glosario

AG Algoritmo Genético.

BiCIAM una biblioteca de clases que resuelve problemas de optimización utilizando metaheurísticas.

COVID-19 Enfermedad por coronavirus de 2019, una enfermedad infecciosa causada por el SARS-CoV-2.

DEAP Algoritmos evolutivos distribuidos en Python (del inglés *Distributed Evolutionary Algorithms in Python*).

HTML lenguaje de marcado de hipertexto (del inglés *HyperText Markup Language*).

HTTP protocolo de transferencia de hipertexto (del inglés *Hypertext Transfer Protocol*).

IA Inteligencia Artificial.

KBO Liga de la Organización Coreana de Béisbol (del inglés *Korea Baseboll Organization*).

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/kbo-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

LMB Liga Mexicana de Béisbol.

LMP Liga Mexicana del Pacífico.

LVBP Liga Venezolana de Béisbol Profesional.

MLB Grandes Ligas de Béisbol (del inglés *Major League Baseball*).

MVT Modelo-Vista-Plantilla (del inglés *Model-View-Template*).

NPB Organización de béisbol profesional de Japón (del inglés *Nippon Professional Baseball league*).

ORM mapeo objeto-relacional (del inglés *Object-Relational Mapping*).

PG Programación Genética.

PTT Problema de Transportación de Torneo.

PVV Problema del Vendedor Viajero.

SSG *Sports Scheduling Group*.

TSP *Traveling Salesman Problem*.

TTP *Traveling Tournament Problem*.

URL localizador de recursos uniforme (del inglés *Uniform Resource Locator*).

© Leslie Pamela Prenes Valencia
Tesis ITAM 2026 · github.com/lesprm/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Referencias

Albert, S. T., Bautista, F. G. J., Vicenç, F. A., y María, S. L. J. (2016). *Programación dinámica.* Universitat Politècnica de Catalunya. Iniciativa Digital Politècnica. Descargado de `https://upcommons.upc.edu/entities/publication/a7f24b45-5fc8-4ed9-94b7-36eeab52b1a4`

Almira, M. H., Rodríguez, R. A. G., Suárez, A. R., y González, L. S. (2021). Sistema para la generación de calendarios para la serie nacional de béisbol. *Serie Científica de la Universidad de las Ciencias Informáticas*, *14*(9), 1–15.

Azteca Deportes. (2022, 02). *¿cuándo arranca la temporada 2022 de la liga mexicana de beisbol?* TV Azteca. Descargado 2022-03-22, de `https://www.tvazteca.com/aztecadeportes/cuando-arranca-liga-mexicana-beisbol-ogr`

Baseball-reference. (s.f.). *Venezuelan league - br bullpen.* Descargado 2022-03-20, de `https://www.baseball-reference.com/bullpen/Venezuelan_League`

Baseball Savant. (2023). *Mlb travel schedule.* MLB Advanced

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/kaiza-tex

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Media. Descargado 2022-02-27, de https://baseballsavant.mlb.com/visuals/map

Baskar, A., y Anthony Xavior, M. (2021). A facility location model for marine applications. *Materials Today: Proceedings*, *46*, 8143-8147. doi: 10.1016/j.matpr.2021.03.107

Borzi, P. (2009, 03). *Think you have a tough job? try putting together major league baseball's schedule and pleasing 30 teams.* Descargado 2023-03-23, de https://www.minnpost.com/sports/2009/03/think-you-have-tough-job-try-putting-together-major-league-baseballs-schedule-and-ple/

Crowe, J. (1987, 07). *The schedule makers : Team doing badly? blame the stephensons of staten island like everyone else does.* Descargado 2023-03-29, de https://www.latimes.com/archives/la-xpm-1987-07-27-sp-3984-story.html

Duarte, G. G. (2019). *Enciclopedia conmemorativa del 75 aniversario de la liga mexicana del pacífico* (1a ed.). Moby Dick Editorial. Descargado 2022-03-20, de https://www.lmp.mx/enciclopedia75

Easton, K., Nemhauser, G., y Trick, M. (2001). *The traveling tournament problem description and benchmarks.* Springer. Descargado 2021-11-08, de https://link.springer.com/chapter/10.1007%2F3-540-45578-7_43#citeas doi: 10.1007/3-540-45578-7_43

Easton, K., Nemhauser, G., y Trick, M. (2002). Solving the travelling tournament problem: A combined integer programming and constraint programming approach. En *International conference on the practice and theory of automated timetabling* (pp. 100–109).

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Gough, C. (2021, 11). *Mlb average attendance 2009-2019.* Descargado 2022-02-24, de https://www.statista.com/statistics/235634/average-attendance-per-game-in-the-mlb--regular-season/

Grootendorst, M. (2021, 02). *9 distance measures in data science.* Medium. Descargado 2022-02-02, de https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa

Holland, J. H. (1992). *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence.* MIT press.

Kariuki, C. (2023, 09). Programación dinámica: Qué es, cómo funciona y recursos de aprendizaje. *Geekflare*. Descargado de https://geekflare.com/es/dynamic-programming/

Kim, T. (2019). Optimal approach to game scheduling of multiple round-robin tournament: Korea professional baseball league in focus. *Computers and Industrial Engineering*, *136*, 95–105. doi: 10.1016/j.cie.2019.07.016

Koza, J. (1992). *Genetic programming: On the programming of computers by means of natural selection.* MIT Press. Descargado de https://archive.org/details/geneticprogrammi0000koza

Koza, J. R. (1998). *Genetic programming ii: Automatic discovery of reusable programs.* MIT Press.

Kuri, Á. (2000). *Algoritmos genéticos.* ITAM. Descargado de http://cursos.itam.mx/akuri/PUBLICA.CNS/2000/AGS.PDF

Luger, G., y Stubblefield, W. (1989). *Artificial intelligence and the design of expert systems.* Benjamin/Cummings Publishing

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Company. Descargado de https://books.google.com/books?id=_IBQAAAAMAAJ

Mercator, P. (2022, 10). *Ellipsoid of revolution oblate (axes aab) with auxiliary sphere.* Descargado 2022-02-06, de https://upload.wikimedia.org/wikipedia/commons/8/8c/Ellipsoid_revolution_oblate_aab_auxiliary_sphere.svg

MLB.com. (2025, 06). *Mlb enjoys 2025 viewership jump in u.s., japan and streaming.* MLB.com. Descargado 2025-10-12, de https://www.mlb.com/news/mlb-viewership-increases-usa-japan-streaming-2025

Nippon Professional Baseball Organization. (2019). *All teams calendar.* Descargado 2022-03-21, de https://npb.jp/bis/eng/2019/calendar/index_08.html

Quiroga, M. (2020). *Investigación de operaciones | 2023 | economipedia.* Descargado 2023-05-24, de https://economipedia.com/definiciones/investigacion-de-operaciones.html

Rameez Naqvi, S., Ahmad, A., M. Riazul Islam, S., Akram, T., Abdullah-Al-Wadud, M., y Alamri, A. (2022). Towards prevention of sportsmen burnout: Formal analysis of sub-optimal tournament scheduling. *Computers, Materials & Continua*, *70*(1), 1509–1526. doi: 10.32604/cmc.2022.019653

Salazar López, B. (2019). Problema del transporte o distribución. *Ingeniería Industrial Online*. Descargado 2023-05-24, de https://www.ingenieriaindustrialonline.com/investigacion-de-operaciones/problema-del-transporte-o-distribucion/

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 | github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Sport League Maps. (2021, 03). *Nippon professional baseball map.* Descargado 2022-03-21, de https://sportleaguemaps.com/baseball/nippon-professional-baseball/

Sports Reference. (2023). *Major league team win totals.* Descargado 2023-05-26, de https://www.baseball-reference.com/leagues/majors

Takeyas, B. L. (2007). Introducción a la inteligencia artificial. *Instituto Tecnológico de Nuevo Laredo*. Descargado de http://www.itnuevolaredo.edu.mx/takeyas

Thielen, C., y Westphal, S. (2011). Complexity of the traveling tournament problem. *Theoretical Computer Science*, *412*(4), 345-351. Descargado de https://www.sciencedirect.com/science/article/pii/S0304397510005451 doi: 10.1016/j.tcs.2010.10.001

Upadhyay, A. (2015, 02). *Haversine formula - calculate geographic distance on earth.* Descargado 2022-03-26, de https://www.igismap.com/haversine-formula-calculate-geographic-distance-earth/?msclkid=8c44e3d8ad5911ec84007dfeac3d59c4

Valverde, A. (2019, 10). *Inicia la liga mexicana del pacífico.* Published by ESTO. Descargado 2022-03-20, de https://www.esto.com.mx/452917-inicia-la-liga-mexicana-del-pacifico-noticias-beisbol-mexicano-calendario/

Voß, S. (2000). Meta-heuristics: The state of the art. En *Local search for planning and scheduling.* Descargado de https://api.semanticscholar.org/CorpusID:16803176

Wikipedia Contributors. (2022a, 04). *Mexican league.* Published on Wikipedia. Descargado 2022-03-14, de https://en.wikipedia.org/

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

wiki/Mexican_League

Wikipedia Contributors. (2022b, 02). *Mexican pacific league.* Wikimedia Foundation. Descargado 2022-03-21, de https://en.wikipedia.org/wiki/Mexican_Pacific_League

Wilson, B. (2020, 07). Mlb releases pandemic-shortened schedule amid testing delays. *AP NEWS*. Descargado de https://apnews.com/article/sports-mlb-health-coronavirus-pandemic-baseball-0c96c8ab67142ca8ab68381c3cdf00b9

Wright, A. (2017, 02). Meet the husband and wife duo behind two decades of mlb scheduling. *Everything Zoomer*. Descargado de https://www.everythingzoomer.com/arts-entertainment/film-television/2017/02/22/mlb-schedulers/

© Leslie Pamela Briones Valencia
Tesis ITAM 2026 · github.com/lespam/kozaitto

Copia de preservación · Prohibida su reproducción sin citar la fuente

Versión pre-oficial

Programación Genética en la Planificación de Temporadas de Béisbol escrito por *Leslie Pamela Brenes Valencia*, se terminó de imprimir en mayo de 2026 en los talleres de Tesis Matozo. Campeche 156, colonia Roma, Ciudad de México.

© Leslie Pamela Brenes Valencia
Tesis ITAM 2026 · github.com/lespam/koza-ttp

Copia de preservación · Prohibida su reproducción sin citar la fuente